如何成为一名合格的程序员

从改需求到造框架

雪碧天 著

目录

自序
飞蛾扑火咏

第一篇　入行

第 1 章 我成为程序员的那天
第 2 章 第一个需求，改到我怀疑人生
第 3 章 第一次上线，我守着日志到凌晨

第二篇　熟练

第 4 章 CRUD 的一年：我很熟练，也很慌
第 5 章 框架是个黑盒，直到它炸了
第 6 章 加班、甲方和“没时间成长”

第三篇　事故

第 7 章 第一次线上事故
第 8 章 锅是怎么找上人的

第四篇　长本事

第 9 章 第一次被人夸
第 10 章 被源码按在地上摩擦的那个凌晨
第 11 章 原来框架是分层的
第 12 章 第一次改框架的代码
第 13 章 第三次写同一段代码时，我停手了
第 14 章 第一个被同事复用的东西

第五篇　造与交

第 15 章 我造了个框架，然后只有我用得顺
第 16 章 只有我能改
第 17 章 我把前端推倒重写
第 18 章 一起把活干完：让东西离开我
第 19 章 尾声（上）：所谓合格
第 20 章 尾声（下）：让英文去编译，让中文去解释
作者简介
光缘志

自序

先说好，这不是成功学，也不是《三天精通某某技术》，更不是“我如何改变世界”的回忆录。真不是。我要有那本事，书名可能就叫《我如何改变世界》了。

我不是一名伟大的程序员，也不是一名出色的程序员。我只是芸芸程序员中的一员。要说经验，也不过是大约 25 年开发工作经历罢了。25 年，听起来挺唬人，其实主要经验是：见过很多错误写法，包括我自己写的。

写这本书，并不意味着我水平有多高，也不意味着我有什么牛气的话要说。我只是把心里一些不吐不快的话，以及这些年经历的一些故事，尽量原原本本地记录下来。它也许是娱乐，但我很认真地对待它。因为只有认真对待它，我的程序员人生才是有意义的。你看，程序员嘛，连自序都得先说明“这不是 bug，是 feature”。

成为程序员，我是幸运的；写代码也是我的爱好之一。我不是优秀程序员，更谈不上伟大，但我和一直走在这条路上的其他程序员一样，朝着同一个方向走。路上，我认识了形形色色的同事、伙伴、leader、CEO——他们很出色。幸运的是，我们都是快乐的，乐于分享快乐的，心无旁骛、勇敢前进的人。看完这本书，你也许会在别人身上看见我，看见你自己，看见过去、现在或未来。要是看见 bug，也别慌，可能是我故意留的。

这本书尝试用脱口秀的风格，以风趣幽默的引线，和读者一起重温那些美好回忆。我一直觉得，好书首先是写给自己看的，我也是这么做的。至于你喜不喜欢，我不知道，因为我不认识你。但我相信一定有人会喜欢，因为他们一定有过类似的经历，也有过同样的感触。如果你不喜欢，也没关系。那说明时间不到；时间一到，你自然就会喜欢它。这话听起来有点像程序员说“在我机器上是好的”，但你别说，还真有点道理。

需要说明的是，本书并非百分之百真实。部分内容借助 Trae AI 协助完成，也有虚构的部分。很抱歉，毕竟我只是一个程序员，不是专业写书的人。专业写书的人负责写书，我负责在书里露怯。

IT 世界里有许许多多牛人。我一直向他们学习，也学着像他们那样思考。我们在不同时期相遇，直到有一天回头一看，已是黄昏。黄昏也挺好，至少不用再开站会了。

AI 时代来了。我看到很多像我这样年纪的程序员，甚至比我们还年轻十岁的，都对 AI 诚惶诚恐。我的建议很简单：AI 是我们的帮手，是我们的工具，是我们的朋友，是在程序员道路上和我们齐头并进的好搭子。感觉对了，问题就解决了。至于怎么用好 AI、发挥它的长处、协助你提升编程能力，我在这里说一声：哥们儿，就看你的了。

飞蛾扑火咏

我本是一只飞蛾，无意干扰人们的生活，只是心向光明，一心扑火；

我就是一只飞蛾，总是坚持自己的选择，明知前途如何，还是要做。

第一篇　入行

第 1 章 我成为程序员的那天

1997 年填志愿的时候，我以为自己报的是一个“做课件”的专业。

华东师范大学，教育技术学。名字很长，长到每次跟人解释都得费点劲：是修电脑的吗？是做 PPT 的吗？解释不清，我干脆说，差不多。就这么含糊了四年。

这个专业的全称，翻译成一句话就是：**教你怎么把知识塞进一块屏幕里，还得让人愿意看。**听着不像程序员，干起来全是程序员。

1998 年，我进了华师大教信系和一家公司合办的实验室，做多媒体、Flash、网站。那是我第一次知道，一个按钮点下去之后，背后要有人写一段东西接住它。也是第一次知道，写这段东西的人，晚上是要加班的。

2001 年，我在上海九鼎待了不到一年；2002 年，我进了上海天络科技，职位叫“互联网软件开发工程师”。技术栈以 Delphi 为主，也用过 Java，做的项目是给某航空公司做培训系统和呼叫中心的 CRM 模块。听着挺唬人。第一天没留下什么具体的事，留下的是一个判断。

公司的人员配置很小，小到不像能接航空公司的项目。可它就在做。我当时的判断很直白：这公司好牛。

我同学那时候已经是项目经理了。

九鼎那不到一年，我遇到第一个朋友，也是我心里第一个师傅。他设计出身，专科学历，爱画画，是那种一眼看得出才华的人。他在考专升本，每天晚上对着一尊买来的雕塑画素描。我以为画一两个小时就完了，他画了两个月，最后画得跟那尊雕塑一模一样。专业和业余的差别，我第一次看明白了：不是会不会，是肯不肯把一件事磨到那个份上。

到了天络，带我的人换成了曹哥——曹操的曹。技术大牛，新东西学得快，学完就能拿来解决眼前的问题。个子高，笑声爽朗，有人格魅力。常带一两个人，就敢做比较大的项目。我在他手下做航空公司的大项目，以 Delphi 为主，也用过 Java。

那年我给自己定了一个判断标准，现在看有点土，但真的管用：**能跑起来，就算入门。**

不是“懂了原理”，不是“看完了教程”，是能跑起来。你写的那行东西，点一下，它动了；再点一下，它没崩。这个门槛低得有点丢人，可它是唯一一个不骗人的门槛。因为“我懂了”可以装，“它跑起来了”装不了。

起点不体面，没关系。这行的起点，普遍不体面。

我见过太多人被“我不是科班”这句话绊住。科班给你的是一张地图，非科班给你的是一条自己蹚出来的路。地图好看，路难看，但走路这件事，最后看的还是脚。

不过我得把话说完整：教育技术学这个出身，也不是全无好处。它教我的第一件事是“用户看不懂，是你的问题，不是他的问题”。做多媒体课件的时候，一个按钮放左边还是右边，一句话写长还是写短，都得替看的人想。这个习惯后来被我带进了写代码里——虽然当时并不知道这算个本事。

坏处也很实在：别人讨论编译原理的时候，你在补课；别人张口就是设计模式的时候，你在想“这词儿啥意思”。我后来补这些课补了很多年，补得也不算好——这是后话，先记账。

我也是半路出家。野路子最大的优势是：你从一开始就知道，东西是给人用的，不是给教科书用的。

优势不能当饭吃。我后来是把欠下的课，一节一节补回来的。

下面这张表，是我给自己找位置用的。你可以拿它对照一下，不用急着往上跳，先看清自己站在哪：

阶段	你大概是这样	一句自检
能跑起来	照着教程能跑通，换个需求就卡	我能不能不看教程，把它再写一遍？
能改对	能定位问题、能改，但说不清为什么	我能不能用三句话讲清这段为什么这么写？

阶段	你大概是这样	一句自检
能说清	能设计，能说服别人，能扛住评审	我的方案被别人挑刺时，我是在解释还是在辩解？
能交出去	别人能接你的活，你不在也不塌	我请一周假，这块东西会不会停？

这张表不是给你打分的，是给你找方向的。我当年在第一行待了很久，久到我以为第一行就是全部。

它还有个用处：**当你觉得自己“什么都不会”的时候，拿它出来看一眼。** 你多半会发现，你不是什么都不会，你只是站在第二行，仰头看着第四行。人一仰头就容易自卑，平视才不会。

门槛低，不等于天花板低。“能跑起来”是入场券，不是终点。真正难的是后面那三段：改对、说清、交出去。它们没有教程，只能在活儿里磨出来。

如果你现在正卡在“我是不是不适合干这行”这个念头上，我想多说一句：这个念头，几乎每个非科班的人都来过，包括那些后来干得挺好的人。它不代表你没天赋，它只代表你还没走够路。判断一个人适不适合，不看他在第一天会不会，看他在第三个月还愿不愿意把手弄脏。

这本书不打算写成一份成功履历。我没什么可炫耀的，真写出来更像是错题本：哪一步想岔了、哪句话听错了、哪个决定当时觉得聪明、三年后想回去把键盘吃了。错题本的好处是，你看的时候不用仰头。

岔开一句：我的八个第一次

说起入行，我能数出八个“第一次”，没有一个是计划出来的。

第一次接触电子线路；第一次遇到“计算机项目工程”这个概念；第一次用 HTML 写出一张能在浏览器里打开的网页；第一次看到 Windows 的窗口，也第一次看到 DOS 那个黑底白字的界面；第一次去考计算机等级，一级考完考二级；第一次用“猫”拨号上网，听那串嘟嘟啦啦的握手声；还有第一次看到学生在计算机上比老师还强。

八件事散在不同的年份、不同的教室里，当时没有一件像是大事。它们潜移默化地把我引上了编程开发这条路——我不是被某个宏大的理由说服的，是被这些具体的小东西一点一点吸过去的。

现在回头看，这八个“第一次”里，没有一个是主动争取来的，都是碰上的。碰上了，我没躲。

本章的一句话：起点不体面不丢人，停在“能跑起来”才丢人。

今天能用的三条：

1. 找一段你一直“照着写”的代码，关掉教程，自己重写一遍，跑起来。跑不起来也没关系，卡住的地方就是你的起点。
2. 给你改过的一行代码配一句中文注释，只写“为什么这么改”，不写“改了什么”。
3. 在手机备忘录里记一条：“今天我在哪一步卡住了，卡了多久。”一周后回看，你会看见自己的形状。

那年我以为，学会用别人的框架就是终点。很多年后我开始造自己的，才知道造东西的人要付什么账——这句话我当时一个字都不懂，也没必要懂。

第 2 章 第一个需求，改到我怀疑人生

装修的时候，业主说：“我这房子也不复杂，简单弄一下。”

你以为简单弄一下是刷墙。真开工才知道，他说的“简单弄一下”，是拆承重墙、改水电，顺便把邻居家的门也换个方向。

我接第一个独立需求，就是这种感觉。2002 年到 2004 年，我在天络，给某航空公司做培训系统和呼叫中心的 CRM 模块，工作内容里有一项叫“需求调研与文档”。这个说法听着文雅，翻译过来是：坐到用户的工位旁边，听他们用行业黑话讲一遍他们的一天，回来变成一堆表和几个页面。

那个需求做的是 CBT 课件，用 Adobe Flash 做，一个个做出来。这个项目我做了一两年。我能很快做出一个个课件，但我没做框架。后来另一个同事进来，从头做了一个框架；大家只管专心做课件，就能接上。

岔开一句：Flash 最早不是拿来做动画的

我用它做了两年课件，从没问过它是怎么来的。

1993 年，Jonathan Gay 和 Charlie Jackson 成立了一家小公司，最早想做的是给一种“能用笔在屏幕上画”的电脑写绘图软件。那种电脑得靠一支笔在屏幕上点，想法挺好，买的人不多；软件的方向也就跟着偏了——他们把它改成做网页上的动画。1996 年底，公司被 Macromedia 收购，那款软件成了 Flash 1.0。在那之前它叫 FutureSplash。后来的名字，是把 Future 和 Splash 拼在了一起：一个公司名，一个软件名。

也就是说，我用来做课件的那东西，起点是一间小公司押错的一个方向。

他们当年要是守着那台笔式电脑，就没有后来的 Flash。

这些跟我做课件没关系。有关系的是另一件事：那两年我做课件做得很快，快到没觉得有什么不对。

我以为我干的是“把话翻译成代码”。干完才发现，我干的是“把话翻译成我以为的话，再翻译成代码，再被退回来重翻”。

这类栽法有个共同的形状：他要的是“一眼看到今天该打给谁”，我做成了一个搜索框。搜索框没错，错在我把“查”听成了“搜”。

我第一次栽，问题不在听岔。第一版是按需求开发完成的；客户看到界面之后，才产生新的想法，又要改。反复改，我就烦了。

栽跟头不止这一种：另一次不是听岔需求，是我根本不知道文档怎么写。头儿（曹哥）给我一套文档清单——需求、概要设计、详细设计、数据库设计、UI设计、测试、帮助，七样，限期一两天；我一片空白，只知道要交出去。他稍微指了几下，先应付过关。他年纪比我小，入行比我早，经验也比我多——我很佩服他。2003 年前后，公司越来越正规，文档也跟着正规；我从这里学会了用文档做团队的协调沟通，也认清了项目留档的分量。再往后，我化繁为简：能用尽量少的内容，在合适的地方放合适的文档。

听懂需求，比写对代码难得多。

难在哪？用户说的不是需求，是抱怨；不是流程，是习惯；不是“我要什么”，是“我现在怎么凑合”。照着字面做，做出来的东西能运行，但没人愿意用。

后来我养成了一个笨办法，笨到不好意思写进简历：**在动手之前，把对方的话复述一遍，让他确认。**

不是“我理解您的意思是……”这种客套，是具体到动作：“您早上来，是先看这张表，还是先点这个按钮？点完以后，您希望屏幕上多出来什么东西？”对方经常会被问住，愣一下才承认：他要的不是这个。

返工的代价，比多数人以为的大。不是多写一遍代码，是脑子里要同时放两个版本的“事实”：你以为的流程，和真实的流程。这两版差得越远，你改得越乱，最后往往不是改对了，是改成了一个新的误会。

所以我后来宁愿在开头多花半小时，也不愿在中途多改三天。半小时能问出来的东西，三天都不一定补得回来。

更难的是承认自己没听懂。多数返工就是从“我懂了”开始的：宁可多改三天，也不肯说一句“我没听明白”——那句话听起来像认输。

这类需求里最常见的坑，不是技术难，是**词太重**。用户嘴里一个词，背后可能是一整套流程。他说“汇总”，可能是“每天下班前算一次”；他说“提醒”，可能

是“三天没动静就打电话”；他说“权限”，可能是“这三个人能看，那两个人只能看自己那一条”。按字面理解，做出来的一定是个“功能齐全但没人用”的东西。

我现在判断一个需求有没有听懂，只看一条：**我能不能说出“他明天早上会怎么用这个东西”。** 说不出来，就是还没听懂，代码写得再漂亮也是白写。

还有一个问题，我后来几乎每次都会问，问完经常能救自己一命：“**如果这次只能先做一个功能，你选哪个？**”对方选的那个，才是真正的需求；剩下的，多半是“顺便也想要”。先把那一个做对，比把十个做一半强得多——做一半的东西，用户不会用；用一个对的东西，用户会告诉你下一个要什么。

需求还有个特点：**说出来的那部分，通常是最容易的部分。** 难的是他没说、也说不出来的那部分——他默认你懂的那些背景。问得笨，比做得错强。让人反感的不是问题多，是你做完一堆之后，问他“这是不是你要的”。

还有件事得替用户说句公道话：他改需求，很多时候不是善变，是他看到东西之后才想清楚自己要什么。人在描述一个不存在的东西时，想象力是有限的；你给他一个能点的东西，他的想法才会长出来。所以“先给个能看的版本”，不只是为了让他确认，也是为了帮他把话说完。

这话反过来也成立：如果你等到全部做完才给他看，那你前面的所有工作，都是在替一个还没想清楚的人猜。猜对了是运气，猜错了是常态。

本章的一句话：需求不是他说了什么，是他要拿这东西去交什么差。

今天能用的三条：

1. 评审时先问三句：谁用、一天用几次；做完拿什么判断它成了；现在没有它你们怎么凑合。问完复述一遍，让对方点头或摇头。
2. 把“验收标准”写进一句话，发到群里或邮件里，留个记录。
3. 先做一个“能看的最小版本”给对方看，别等全做完再端上去。

那种感觉我倒是记得：不是我不会写代码，是我不知道自己在给谁写。

需求搞定了，但我发现真正的麻烦在代码之外——比如“上线”两个字。

第 3 章 第一次上线，我守着日志到凌晨

第一次独立上线，像第一次一个人开车上高速。

平时练车都有副驾，副驾上的人会替你踩刹车——虽然你嘴上嫌他烦。真一个人上了高速，你才发现，方向盘这么轻，路这么窄，隔壁的车这么快。你不敢变道，也不敢减速，因为后面有车。

我第一次上线那天，情况差不多。代码是我写的，测试是我点的，打包是我打的，到了晚上，所有人都走了，剩下我和一台机器。我盯着日志看，像盯着一锅还没开的汤。

我在天络做的主要是 Delphi 客户端，给某航空公司做培训系统和呼叫中心的 CRM 模块。那个年代的上线，不像现在点一下流水线就完了，得把东西装到人家的机器上，得跟现场的人约时间，得考虑“装完之后万一连不上怎么办”。

代码写完不算完，上线了才算开始。

这句话听起来像废话，但它是分水岭。写代码的时候，世界是可控的：输入输出、分支循环、异常捕获，都在你的脑子里。上线之后，世界就不可控了：网络会抖，数据会脏，用户会按你没想到的顺序点，别人会在你没想到的时间点重启一台机器。

所以上线的第一课不是“怎么装上去”，是“怎么退回来”。

后来我做事，先问自己三个问题。问了很多年，问成了习惯：

1. **出事了怎么退？** 有没有一条能在十分钟内回到昨天的路，谁有权执行，执行完谁知道。
2. **出事了谁在看？** 是“我今晚守着”，还是“有个地方会响”，别把系统安全挂在一个人的睡眠上。
3. **怎么知道它坏了？** 是用户打电话骂你，还是你看得见一个数字在往下掉。

这三个问题，我在第 7 章吃过大亏——那一次，三个答案都不太好看。这是后话，先按下。

有一件事我到现在都记得：**上线之后的那几个小时，人是不自由的。** 你人在家里，心在日志里；手机一响，你先看是不是报警，不是报警才敢接。

后来我学会了一件事：把“守着”变成“看得见”。不是人不守，是不用靠人肉守。

这话说起来轻巧，做起来得靠笨功夫。你得先知道“正常”长什么样，才知道“不正常”是什么样。上线前，我会在纸上写三个数字：今天大概多少人用、大概多少笔业务、报错大概几条。写完这三个数字，晚上盯盘就不是干瞪眼，是拿眼睛去对答案。对不上，就先别睡。

也正是从那几次开始，我有了“上线前写一页纸”的习惯。不是公司要求的模板，是自己给自己写的一张便签：今天动了什么、可能会影响什么、出问题先看哪里、找谁。写完这张纸，我心里就有底了——不是因为问题不会来，是因为问题来的时候，我不用现场想。

后来我把这张纸变成了清单，再后来变成了团队的习惯。它的价值不在于“全”，在于“提前想过一遍”。提前想过一遍的人，出事的时候手不抖。

上线之后的第一小时，我一般只干三件事：看核心数字、看错误日志、看有没有人在群里喊。这三件事的顺序是有讲究的——先看数字，因为数字最诚实；再看日志，因为它会告诉你“哪里不对”；最后看群，因为群里的话往往已经晚了半步。

第一小时过去了，如果一切正常，我才敢去做别的事。不是因为我谨慎，是因为我怕“以为没事”这四个字。系统最会骗人的时候，就是它安静的时候。

还有一句话，我后来常跟带的人说：**上线不是你一个人的事。** 你觉得是你按了那个按钮，其实是有人在后面配环境、有人在前面接用户、有人在事后帮你兜。你可以紧张，但别把紧张变成“谁都别碰”。一个人守着的上线，看着负责，其实最脆弱。

那个年代没有现在这些监控面板和流水线，很多事只能靠人盯、靠电话、靠一句“好了没”。我不怀念那种苦，但我知道它教会了我一件事：**看得见，比守着强。** 工具变了，这句话没变。

本章的一句话：上线不是提交，是交接——把东西交给一个你不在场也会运转的世界。

今天能用的三条：

1. 退路：上线前把回滚步骤写成三行贴到群里——谁来做、多久能回到昨天。
2. 眼睛：上线后盯的不是“有没有报错”，是核心那个数字有没有变；先弄清“正常”长什么样。
3. 嘴巴：出事先通知谁、说什么、多久说一次；再给自己定一个“下线时间”，几点之后不再动代码，只观察。

那晚没出事——也可能出了，我不知道。但我后来知道，出事只是时间问题。

第二篇　熟练

第 4 章 CRUD 的一年：我很熟练，也很慌

流水线工人有个特点，我在自己身上看见了：手比脑子快。

一个需求过来，不用想，手已经开始动了。建表、写接口、拼页面、联调、改字段、再加个状态。键盘敲得很响，屏幕滚得很快，一天下来人是空的。你问我这天干了啥，我说“做了三个需求”，再问具体做了啥，我得翻提交记录。

2004 年到 2006 年，我在逊志，写 Java，做中文问答平台的核心开发，也做 Linux 上的部署。那两年我最大的收获不是技术，是熟练。熟练到什么程度？业务方说一半，我就知道他要什么字段；接口文档不用看，我知道返回结构长什么样。

逊志的办公点在宜家家居那栋楼。公司体系是成熟的：运营、架构、测试各有分工，同事也都很出色。我在那儿开始深入用 Java 做 Web 开发。手上有一个独立项目：开发一个类似 MSN 的聊天机器人。此外也做过增值服务。

熟练不等于成长。流水线能把你练熟，也能把你用废。

这话不是骂流水线，是骂我自己。熟练本身没错，错在熟练之后那种“我已经会了”的舒服感。舒服感一上来，人就不问了。不问为什么这么分层，不问为什么这个字段要有默认值，不问为什么这段代码三个月前被别人改过一回。

后来我确实慌过一次。没有具体的事故，也没有谁骂我——就是某个再普通不过的时刻，我突然发现自己说不清“为什么这么写”——我只知道“以前就是这么写的”。那一刻有点像开车开到家门口，突然想不起来刚才那十分钟是怎么开过来的。

我没有立刻辞职去读源码，也没有立刻开始写框架。我只是做了件很小的事：每周挑一处“我只会照着写”的地方，去查一遍它为什么这么写。

查着查着就发现，很多“规定”是有原因的，只是原因没写在文档里；也有很多“规定”根本没原因，就是前人顺手写的。这两类东西混在一起，长得一模一样。

这时候最危险的一句话是“一直就是这么写的”。它不是理由，它只是把问题往后传。你今天不问，明天接手的人也不敢问，最后这一整块东西就变成了没人敢碰的祖传代码——它不一定坏，但也没人知道它为什么好。

所以我后来给自己定了个很低的门槛：**每接手一块陌生代码，至少弄明白一个“为什么”。** 一个就够，不求全懂。弄明白一个，你就有了一个可以撬动整块代码的支点；一个都没有，你就只能照着抄。

熟练期还有三种假象，我全中过：

- **快，就是强。** 手快只说明你路径熟，不说明你路径对。
- **不报错，就是稳。** 不报错可能只是没人走到那条分支。
- **别人不问，就是懂。** 别人不问，多半是懒得问，或者已经不指望你了。

这三种假象的共同点是：它们都很舒服。舒服的东西，通常不让你长本事，只让你长年纪。

更麻烦的是，它们还会给你正向反馈。你手快，组长夸你效率高；你不报错，测试说这块挺稳；别人不问，你就默认自己讲清楚了。反馈是真的，结论是假的——你拿到的夸奖，是流水线的夸奖，不是你的。

那怎么把熟练换成成长？我的办法很笨：**每做完一个需求，逼自己回答一次“这次我多会了什么”。** 如果答案是“更熟了”，那这次就没有成长，只有消耗；如果答案是“我知道了原来这个字段不能这么用”，那才算攒下一点东西。

这个答案不用写得多漂亮，一句话就行。攒够十句，它们连起来就是你自己的一本小册子。不是别人教的，是你自己撞出来的。

也有人说，那我换个环境、换个技术栈，是不是就不慌了。我的经验是：**换个环境能换掉腻，换不掉慌。** 慌的根不在你写什么语言，在你不知道自己写的东西为什么值得存在。这个问题的答案，换几家公司都长不出来，只能自己想。

不过我得承认，那两年的熟练也不是白来的。它给了我一样很实用的东西：**模式识别。** 看到一个需求，我能立刻想出它会长成哪几张表、会踩哪几个坑。这东西不高级，但它是地基——没有它，后面那些“设计”“抽象”全是空中楼阁。地基不丢人，丢人的是拿地基当房子。

地基和房子的区别，说穿了就一句：地基是“我知道这里会出问题”，房子是“我知道为什么这里会出问题”。前者靠次数堆，后者靠脑子想。

本章的一句话：熟练是手快，成长是嘴能说清——能说清“为什么”，才算真的会。

今天能用的三条：

1. 每周挑一处“照着写”的地方，查文档或问人，弄清它为什么存在。
2. 把一个重复了三次的动作，抽成一个小脚本或小工具——哪怕只省你十分钟。
3. 在备忘录里写一句“这周我搞懂的一个为什么”，不写感受，只写原因。

那阵子我以为，只要再熟练一点，慌就会消失。后来发现，慌不会消失，它只是换了个问题：从“我会不会写”，变成“我写的这些东西，凭什么值得存在”。

慌归慌，我不知道从哪下手。直到框架先对我动了手。

第 5 章 框架是个黑盒，直到它炸了

框架这东西，特别像一台全自动洗衣机：按两个键就行，用上两年，人会觉得它是个好人。

直到有一天它不转了。按两个键，没反应；再按两个键，还是没反应。站在它面前，你才会第一次意识到：我不知道它里面长什么样。

框架就是那台洗衣机。平时按两个键，它把活干了，双方配合得挺好。等它不干活的那天，才发现这不是配合，是单方面使唤一个自己完全不了解的东西。

我在优孚的时候，第一次认真用 SilverStripe——一个当时身边没人用、文档也不算厚的 PHP 框架，从选型到投入生产，基本靠自己摸着来。

把我拉进优孚的是同学。他在公司做高管，能直接跟老板、跟 CEO 说话。他找我去，是因为研发。第一件事是攻一个国外客户。对方用 SilverStripe，国内几乎没有可参考的经验，也几乎找不到会的人；我进去替公司把这套东西奠基。第二件事是政府的一个办公平台，我主导，用 Adobe Flex 加 PHP 做。

对接的是一个外国人，我们用 Skype 沟通。项目做到能投用之后，他送了我一本 SilverStripe 的书，纯英文的。那种被欣赏的感觉很具体：我有点成就感。他还让我意识到一件小事——代码里空格和 Tab 别混着用。这跟跑不跑得起来没关系，跟别人读起来顺不顺有关系。

岔开一句：这套框架是从一间小办公室长出来的

送我书的那位，大概没想过我会去查这套框架的来历。

SilverStripe 是 2000 年 2 月在新西兰惠灵顿成立的，三个创始人——Sam Minnée、Tim Copeland、Sigurd Magnusson。2006 年，他们把这套建站系统按开源许可放了出来——说白了，代码公开，谁都能拿去用、拿去改。后来新西兰的政府机构和一些公共机构，也用它建站。

一套建站系统，就是别人搭好的骨架，你在上面填东西。这副骨架是三个新西兰人搭的。

我记住的是另一层：它在南半球，离我这儿很远，文档也不厚，可它确实被一些正经机构拿去用了。

这对我有一点安慰：小地方、小团队做出来的东西，也能被正经场合用上——我在优孚干的，差不多也是这个位置。

书我收了，也翻了。翻完我只有一个念头：这套东西，我得自己上手试一遍。

摸着来的意思，不是“我看了文档就会了”，是“文档没写的部分，我靠猜”。猜对了，项目往前走一步；猜错了，报错信息看不懂，搜索引擎也搜不到——因为没人用过，所以没人写过。

那种感觉很具体：页面白屏，日志里一句看不懂的报错，文档里通常只有一句很客气的话——大意是“这个方法很灵活”。灵活是个很客气的词，翻译过来就是“出事了自己想办法”。

把框架当魔法的人，迟早被魔法惩罚。

这话说得重，但它是真的。框架不是魔法，它是一堆别人写的规则。规则写清楚了，你觉得它好用；规则没写清楚，或者你没读到，它就变成了玄学。玄学最坑的地方不是它错了，是它错了你不认识。

后来我形成了一个很土的三步：**复现、隔离、找证据。**

- 复现：先让它在你手里稳定地错一次。不能复现的问题，等于不存在，你只是在跟它赌气。
- 隔离：把无关的东西全拿掉，剩下最小的那段。十有八九，问题不在你以为的地方。
- 找证据：日志、断点、文档、源码，四样里至少拿一样。猜出来的原因，不算原因。

这三步里，最费劲的是第一步。人在着急的时候，最不想干的就是“再让它错一次”——你想直接跳到答案。可跳过复现的代价，通常是把一个问题变成三个问题：你改了一处，错误变了，你以为是新问题，其实是老问题换了个姿势。

复现还有一个副作用：它会逼你把情绪换成步骤。写“怎么操作会错”的时候，你没法写“它怎么能这样”，只能写“第一步、第二步、第三步”。写着写着，人就冷静了。冷静不是修养，是流程的副产品。

读源码这事，后来我也摸出一点门道：不要从第一行读。先从报错的那一行往上追，追到“事情开始不对劲”的地方，然后停下来，把那段读懂。读懂一段，比通读一遍有用。通读是给自己一个“我看过了”的安慰，追读才是真的在找东西。

这套办法不快，但它有个好处：下次再遇到同类问题，你脑子里有路径，不是一片白。

框架的问题有个规律：它很少在你闲着的时候来。它总在你赶版本、换环境、加机器、改配置的时候来——因为那些时候，你动了他平时不动的东西。所以我现在换环境之前，会先问一句：“这次我动了什么？”答案往往就是问题的方向。

这不是什么高深的方法，就是承认一个事实：**大部分“框架的锅”，其实是“我们改了它没预期的事”。** 承认这一点，排查范围立刻小一半。

黑盒不可怕，可怕的是你假装自己懂。假装懂的成本很低——你可以在会上点头、在文档里复述、在代码里照抄；但它的利息很高，等到某天它不转了，你要连本带利地还。

所以我对框架的态度一直有点矛盾：用的时候希望它别烦我，出事的时候希望它别藏。这两种希望，最后只能靠一件事满足——**我大概知道它在干什么。** 不用全知道，知道主干就行。

第一次点开框架源码那天，我以为会看到一副骨架，结果看到的是类套类、接口套接口，像走进一个别人的家，灯是亮的，但你不知道开关在哪。我看了二十分钟，关掉了。不是看懂了，是看晕了。

真正把我逼进去的，是后来那次事故——出在 jHipster 上，一套 Java / Spring 的生成器。那次不是白屏，是一个活动被卡到跑不动——这是第 7 章的事，这里先记一句话：**框架救不了你，它只会让你错得更整齐。**

本章的一句话：能复现的问题才叫问题，剩下的都叫情绪。

今天能用的三条：

1. 复现：写下一页“怎么操作会错”，让别人也能照着错一次。
2. 隔离：删掉一半代码或配置，看还错不错；再删一半，直到剩下最小的一块。

3. 找证据：日志、断点、文档、源码，按这个顺序找，找到一条能解释现象的证据为止。

第 6 章 加班、甲方和“没时间成长”

挤地铁的时候，你会有一种错觉：挤上去就赢了。

真挤上去才知道，赢的只是“上去了”，接下来你还是站着，手抓着扶手，脸贴着别人的背包，一站一站地熬。加班也是这样。你以为忙完这一阵就能喘口气，忙完发现，下一阵已经在门口等着了。

我做甲方的项目，最直观的感受是：**你的时间不由你安排。** 需求是对方提的，评审是对方开的，上线时间是对方定的，连“这个字段能不能加”都要等对方内部开完会。你在自己的公司上班，但你的日历属于别人。

在天络那几年，我做过需求调研。需求调研就是去现场看人家怎么干活：怎么接电话、怎么记工单、怎么在系统里翻半天找一个客户。看完回来写文档，写完再对着文档做。做出来再拿回去给人家看，人家通常会说，这不是他们平时的做法——于是再改。

忙到什么程度，各人的尺子不一样。

但我可以说一个不必编的感受：**忙的时候，人是没有“成长时间”的。** 成长时间不是公司给的培训，也不是周末的自学计划——那些东西在忙季第一个被砍掉。真正能长出来的东西，只能从活儿里顺手攒。

我管这个叫“顺手攒”。

顺手攒的东西，一开始都很小，小到不值得跟人提。比如一张自己画的表结构草图，比如一段复制了八百遍的 SQL，比如一句“这个接口有个坑，别在循环里调”。它们不算成果，不能写进周报，也不好在面试里讲。但它们会在某个具体时刻救你一次，然后你就再也离不开它们了。

反过来，那些“等我有空再系统学”的计划，我做过很多次，几乎没有一次活过两个月。不是意志力的问题，是时机的问题：你把学习安排在一个“本来就不存在”的空档里，它就必然被现实吃掉。

所以我现在不太相信“学习计划”，我更相信“学习寄生”——把要学的东西寄生在一件必须做的事上。要做报表，就顺手把 SQL 写规范；要改接口，就顺手把

旧接口的坑记下来；要修 bug，就顺手写个最小复现。事情做完了，东西也留下了。

一个需求做完，别人合上电脑就走了。我多留十分钟，把这次踩的坑写成三行：什么现象、什么原因、下次怎么绕。写完就扔进一个文件夹，再也不看。三个月后再遇到同类问题，翻出来，能省下不少时间。

一个重复动作做了三遍，我就顺手把它改成脚本或模板。第一次可能多花二十分钟，之后每次省十分钟，做几次就回本。这笔账很土，但它真的是笔账。

一个字段为什么要加、一段逻辑为什么不能删，我写进提交信息里。不是为了给谁看，是为了三个月后的我——那个我，一定不记得今天。

时间不是挤出来的，是顺手攒出来的。

这话有个前提：你得先承认，忙是常态，不会因为你努努力就消失。承认之后，你才不会把“等我闲下来再学”当成计划。等闲下来，是这行最贵的谎话。

还有一层更现实的：在甲方项目里，你的“完成”和他们的“验收”经常不是一回事。你觉得做完了，他们说“还要再改改”；你觉得改好了，他们说“我们再内部讨论一下”。这不是他们坏，是他们的流程就长这样。你能做的，不是抱怨，是把“什么算完成”提前问清楚，把每一次“再改改”记下来——记着记着，你就知道他们的验收标准藏在哪句话里了。

忙季怎么保住这点“顺手”？我的做法是把它塞进已有的动作里，不另开时间：提交代码的时候多写一句为什么，关需求的时候多留三行坑，写周报的时候多写一句“这周我搞懂了什么”。这三件事本来就要做，只是多数人做得很快、很空。你稍微慢十分钟，它们就变成了你的东西。

也说句实话：加班能换来交付，换不来本事。交付是公司的账，本事是你自己的账。两本账可以一起记，但你得知道自己在记哪一本——不然忙了三年，账上只有别人的数字。

这不是劝你别加班。该加的班躲不掉，我只是想让你在加完之后，还剩下点自己的东西。

本章的一句话：把需求当练手素材，你就不用等“有空的时候”。

今天能用的三条：

1. 今天下班前，写一条“现象—原因—绕法”，扔进一个固定文件夹。
2. 找出你这周做了三遍的同一个动作，把它变成一次性操作。
3. 下一次提交，别写“修改 bug”，写清楚“为什么”。

那几年，我以为自己攒下的是经验。后来才知道，攒下的是小聪明——能用，但不值钱。

真正的耳光，在一个普通的晚上扇了过来。

第三篇　事故

第 7 章 第一次线上事故

一、我以为我干了件漂亮事

雪橇科技。山东。一场爱闯闯的活动。

活动开始前，我提交了一段改动。

改动不大：往一个定时任务里，加了一个循环体。一行到不超过十行。

“一行到不超过十行”——这句话我现在听着耳熟。它不是一次汇报，是一句台词。

提交之前，我盯着屏幕看了很久。心情接近一位老中医给自己开的方子：补气、活血、还便宜。

这段代码跑得不算慢。但我知道它能更省。

我甚至给它起了个名字，叫“性能优化”。名字先到位，事实慢慢补。

为什么动它？我到现在都记得那个判断：这段原来的代码比较烂，我能优化它，能提升性能。

注意这句话的顺序。“烂”是先出现的。

不是“这里有问题”，是“这里写得不行”。

这两个说法差别很大。前一个需要证据，后一个只需要你比写这段代码的人多懂一点点。

“烂”字一出口，后面的事就顺理成章了。既然烂，那就顺手改一下。一行到不超过十行，能出什么事。

老中医那个比喻，还得补一句：他之所以敢给自己开方子，不是因为懂医，是因为他觉得别人开的方子不行。

再说一句不太好听的：没人要求我改。

我最擅长的，就是在没人问的地方，证明自己懂得多。

二、没有报错，没有告警，只有“越来越慢”

手机 APP 越来越慢。

不是白屏。不是报错。不是谁收到了一条告警。就是慢。

刷新一下，等得久一点。再刷新，再久一点。

到最后，整个活动卡到无法运行。活动办砸了。

后来我把因果捋直了。定时任务不是执行一次就完事的东西，它每一轮都会重来一遍。我在里面加了一个循环体，等于每一轮调度都多做一遍工作量。

多一遍。再多一遍。一轮一轮往上叠。

系统不是被我一下推倒的，是被我一天一天压沉的。

定时任务有个特点：它不懂累，它只会准时。

而这种“越压越沉”的故障，是最难被发现的。它没有报错，没有红灯，没有一条能指着说“就是这里”的日志。

所有监控都在盯一件事：有没有断。

没有一块屏在看另一件事：是不是越来越沉。

我们给系统装了心电监护，只会报警“心跳停止”，不会报警“这个人正在变胖”。

最吓人的不是炸掉。炸掉有现场。

慢慢地不行了，只有一句谁也说不清的“最近有点慢”。

这里得说一句不好笑的实话：这次事故的代价，不是我丢人。

是一场办砸了的活动。是别人的活动、别人的钱、别人的周末。

三、我那天想通的一件事

后来我把那段改动翻来覆去地看。

它没错。真的没错。逻辑是对的，在小流量下它也确实是快的。

这才是最难受的地方：一个没错的改动，把一场活动弄没了。

如果它错了，我还能骂它。它没错，我只能骂自己。

问题在另外三件事：我从来没量过它到底快了多少；我从来没在真实峰值下试过它；我也从来没给自己留一条能退回去的路。

还有第零件事，更根本：我改的是一个会重复执行的东西，却按“改一次”来评估它。

定时任务、消息消费、拦截器——这类地方的改动不会只发生一次。它会按调度的节奏，替你把同一个判断重复很多遍。

这个道理不难。难的是当时我压根没往这儿想。

所谓优化，一共就三层：它现在多慢、它最慢的时候有多慢、它不灵的时候怎么撤。

我当年只做了最前面那一层——我觉得它烂。

我以为我在做优化。我其实在做赌。

赌注不是我的。

所以我这章想说的判断是这一句：

性能优化不是把代码写得更聪明，是先证明它更快，再保证它坏不了。

再短一点、能被复述的版本是：

量不出数字的优化，不叫优化，叫许愿。

许愿这件事，我们这行做得特别专业：嘴上叫优化，心里叫祈祷；提交信息我写得很快，快得像在给自己批假。

四、许愿之前，先做三件事

这三条是我后来给自己定的，用到现在。不多，就三条。

一、先量。

改之前先拿到基准数：压测数据，或者线上真实流量的采样。说不出“从多少改到多少”的改动，不叫优化，叫手感。

二、先压。

在接近真实峰值下跑一遍。而且不要看平均值，看最慢的那 5%。线上的卡顿从来不是平均值惹的，是那批最慢的请求堆出来的——平均值好看，是因为绝大多数请求都没赶上你最忙的那一秒。

这一条对我这次的教训还多拐了个弯：如果改动加在会重复执行的地方，那要压的不是“改完之后快不快”，而是“它重复很多轮之后还快不快”。

前者是一天。后者是一个月。

三、先备。

上线前留一个开关，能在一分钟内退回旧逻辑。退不回去的优化，不许上。

回滚方案不是给“万一会出事”准备的，是给“我一定会判断错一次”准备的。

如果你手上的团队确实做不了完整压测，退一步：至少用小流量灰度，跑够一个完整的业务高峰，再全量放开。

退路可以简陋，但不能没有。

这三条的共同点是：它们都不高级。压测不高级，加个开关不高级，看最慢的 5% 也不高级。

高级的是我当年那句“这段代码比较烂”。

还有一点：这三条都不需要聪明。

五、收口

这件事过去很多年，我唯一没改掉的习惯是：现在任何一次自以为“优化”的改动，我都会先在纸上写三个数——改之前是多少、我打算改成多少、慢到什么程度我就认输。

写完这三个数，一半的“优化”自己就取消了。

它们本来只是想让我显得聪明。

至于我的主管，就叫他 Loneway：这件事的责任，他自己担了，什么都没说。

不是事后找我谈了一次话，也不是拍了我的肩膀说“下次注意”。就是一个字都没提。整个过程里，Loneway 照常开会、照常派活，好像那件事没有发生过。

要是他骂我一顿，我大概早就翻篇了。

他没骂。于是这件事在我这儿，一直没结案。

每次我想偷个懒、觉得“这点改动能出什么事”的时候，那段沉默就会准时出现。

这里再补一句不好笑的实话：我不知道 Loneway 当时怎么想的。我也不打算替他想。

这次是别人替我挡的。下一次，会有人替我挡吗？

再往后我发现，难的从来不是“不出事”，而是有一天，你会主动去碰一个没人替你兜底的东西。

本章的一句话：性能和代码一样，能证明的才叫存在。

第 8 章 锅是怎么找上人的

一、锅不认谁干的，只认谁在场

小时候我以为，背锅是被冤枉：一口锅从天而降，砸在背上，回头一看，扔锅的人已经跑了。

工作几年我才明白：大部分锅不是扔过来的，是本来就放在那儿。谁离得近，谁就背上了。

这条规则很省事。它不看“谁干的”，只看“谁在场”。人在，离得近，手上正好有那把钥匙，锅就归他。至于这事当初是谁决定的、谁省的测试、谁压的排期——锅不认这些。

第 7 章那场事故，责任 Loneway 自己担了，一个字没说。我当时以为，这就是“责任”最体面的样子。

我只看见有人扛着，没看见那口锅本来落在哪儿。我有个毛病：看见别人扛事，先感动，后复盘。

后来才知道，那只是事情的一面。另一面是：锅要落到谁头上，是有路径的。那次事故的经过第 7 章已经讲过，这里只说路径。

二、锅分三种，会上只聊一种

复盘会这种场合，有个很稳定的规律：**第一句话永远是“谁改的”。**

为什么总是三个字？因为它最便宜。不需要证据，不需要上下文，只需要一个名字。名字一出口，会议就有了结论的形状，大家就可以散会了。

我承认，三个字一出来，我也会先在心里过一遍：那几天我碰过什么。

但名字不是原因。名字只是“最后一个碰它的人”。

我后来想明白的事，也是这一章想说的：

锅的背后，通常站着一个没想清楚的设计。

“谁改的”只能回答最后一环。往前看：这个改动为什么能一路通过？有没有人看过？有没有灰度？有没有一条告警能看见它？出事之后能不能在一分钟内退回去？

这些问题一个都答不上来的时候，锅的形状就已经定好了。它只需要一个恰好站在那个位置的人。

所以还有一句更不客气的：

背锅的人，通常不是最该负责的人，是最靠近故障的人。

这话在会上说出来，效果通常不太好。因为它听起来像“不是我的错”。而“不是我的错”这五个字，在别人耳朵里，约等于“我不打算改”。

再往前一步，锅其实分三种：

- **技术锅**：代码写错了、配置配错了。最好认，因为有日志。
- **流程锅**：没人评审、没有灰度、没有回滚。最难认，因为它的证据是“一直都这样”。
- **沟通锅**：两个人各理解了一半，谁都没说出来。最冤，因为它连记录都没有，只存在于“我以为他知道”。

有意思的是，会上讨论的永远是第一种。不是大家坏，是第一种能查，后两种只能靠人回忆。而回忆这件事，在会议室里最容易变形。

三、先说时间线，不说人

我后来给自己练了三句话，开会时用。不好听，但省事。

第一句：先说时间线，不说人。

“几点开始不对、几点发现、几点恢复、影响了谁。”事实摆完，情绪自己会降一档。很多会开到一半变成形容词大赛，就是因为没人肯先把名词说完。

第二句：认自己那一段，别认整口锅。

“这个改动是我提交的，我负责的是 X；Y 那一段要从 A 那边看。”这句话的难点不在措辞，在得真的知道自己负责哪一段。说不清的人，最后只能整口锅都背着。因为拒绝不了，也说不出边界。

第三句：把锅变成一条能检查的规则。

“下次这个位置加一条告警 / 一个开关 / 一次灰度。”复盘会开完，如果没产出一条能检查的东西，那这场会只是把情绪重新分配了一遍，明天照旧。

还有一句前置，我后来才加上：**先确认自己到底错在哪一段。**

因为“我错了”这三个字说得太快，会变成一种偷懒：用态度换免于追问。姿态越低，问题越容易被放过；被放过的那个洞，下次还会在。

态度解决的是气氛，不是问题。

（这三句话我在心里排练过很多遍。真到会上，第一句往往还是“这个……我觉得……”。所以我把第三句写在纸上，会开完就发出去。嘴靠不住的时候，让纸靠得住。）

复盘会最后五分钟还得落一件小事：指定一个人、一个日期、一条验收标准。没有这三样，那条“规则”活不过一周。下次开会，大家会重新吵一遍同样的内容，只是换了个人背。

四、那个位置，本来没留第二个人

复盘会我参加过不少。

但“开完会”和“想明白”是两件事。

我一开始只想明白了一半：锅按位置分，谁离得近，谁就站在锅底下。

另一半是很多年以后才想明白的：锅之所以总能找到人，是因为那个位置上，本来就没有第二个人。

不是别人的问题，是设计的问题。一个系统如果只有一个人知道怎么改，它出事的概率不会因为这个人更努力而降低，只会把出事的时间往后推。

不是别人不肯站，是那个位置从来没给第二个人留过入口。这一层，后面还有一整章要说。

这个道理不难，我明白得却不算早。等我明白过来，我已经在那个位置上站了很多年。

散会之后没多久，我居然被夸了。而且被夸的原因，挺让我意外。

本章的一句话：锅不是扔过来的，是设计漏出来的。

今天能用的三条：

1. 开会前先把时间线写在纸上：几点、什么现象、影响谁、几点恢复。只写名词，不写形容词。
2. 会上用一句“我负责的是 X，Y 要从 A 那边看”，把自己那一段划出来——先划边界，再谈感受。
3. 散会前逼出一条能检查的规则（告警、开关、灰度、工单任选一），发进群里，别只留在会议记录里。

第四篇　长本事

第 9 章 第一次被人夸

一、夸奖像食堂的免费汤

公司里的夸奖，很像食堂的免费汤。

不顶饱。但你正好渴的时候，它挺管用。

我喝过的那一碗，多数时候不是一句话。是对方的眼神突然一亮——那一下，我心里会冒出一句：这哥们牛啊。夸我的人主要是同事、伙伴、甲方等等。它不是评价。评价要走流程，这个不用。

但有个判断可以先摆出来：那件事，多半不在我的任务清单上。

这就有点意思了。清单上的活，干得再好，评价一般是“完成了”；清单外的活，只要做对一次，评价就变成“没想到你还管这个”。

同样是一小时，写在任务清单里，是本分；落在清单外，是惊喜。

人不是按工作量给评价的，是按“预期差”给评价。

食堂的汤还有个特点：喝的时候没人看你，喝完也没人问咸不咸。

夸奖大多也是这样。它不改绩效，只改那天的心情——而心情这东西，最容易让人多做一件本不该接的事。

二、被夸，是因为补了一条缝

所以这一章最想说的判断是：

你被夸的那次，往往是因为做了没人要求你做的事。

不是因为更聪明，也不是因为更努力——加班的人多了，没人给加班发奖状。是因为补上了**分工之间的那条缝**。

分工这个东西，写在岗位描述里、写在排期表里、写在每个人的 OKR 里。但缝不写在任何地方。缝是“这个接口谁来兜底”“这份数据谁来核对”“这个报错谁来解释”——所有人都以为有别人管，实际上没有。

缝的特点是：它在的时候没人看见，它裂开的时候所有人都疼。补上它，大家的第一反应不是“谢谢”，是“哦，本来就应该有”。所以分外之事的天花板很低——干到最好，也只值一句“挺好的”。

但它有一个隐藏利息：**疼过的位置，只有补的人知道。**

很多年以后，关于我给公司做的那几套系统——官网、帮助、报表——我写过一句总结，大意是：“它们虽然不重要，但是却大大地减少了公司的工作量。”这句话里有两个词要一起看：不重要、大大地。放在一起，就是辅助系统的全部处境：省下来的时间是真的，被记住的程度是零。

（这句我只放在这儿，不展开——第 15 章专门说它。）

为什么缝一直没人补？原因很现实：补缝不产出可汇报的东西。周报上写“本周完成三个需求”很体面，写“本周让系统少疼了一次”像句废话。所以缝永远缺人，而补缝的人永远不缺活。

再补一层反常识：

分外之事有两种。一种补的是系统的缝，一种补的是自己的焦虑。

前者做完，系统少疼一次，人也轻松一点；后者做完，只是把自己变成一个更忙的人，而且没人认。区别很好判断：做完之后，是不是少了一件**必须亲自处理**的事。如果反而多了一件，那多半是第二种。

三、做的人少，缺了会疼

识别“分外之事”，我只有一个标准，一条就够：

做的人少、缺了会疼。

两条都满足，才值得动手。只满足第一条，那是冷门；只满足第二条，那是大家正在忍着——忍着的东西，不一定需要有人来救。

还有一条边界，得说清楚：**分外之事别做到别人的职责里去。** 补缝是补没人管的地方，不是替别人干他该干的事。前者做完，关系更好；后者做完，你成了那个“爱插手的人”，而且出了事还是你的。

这些认识都来得晚；最值钱的，往往也是最后才想明白的那一点。

四、被夸的副作用，是飘

被夸这件事，对我最大的影响不是高兴，是**飘**。

飘的方式很具体：开始觉得，“没人做”的事，都该我做；“没人会”的事，都该我会。

我当时把这种飘理解成上进。

这跟第 7 章那个老中医是同一种病——换个科室，又坐了一次诊。他敢给自己开方子，是因为觉得别人开的方子不行；我敢什么都接，是因为觉得别人接不住。

而且“被夸”还有一种更隐蔽的副作用：它会把你钉在那个位置上。第一次是你顺手做的，第二次是大家默认你做，第三次你不做，就成了“你最近是不是不上心了”。

所以这一章的结尾，不是“我从此更谦虚了”。

然后我干了一件让自己疼了很久的事：我点开了框架的源码。

本章的一句话：被夸的那次，通常不是分内的事，是替系统补的那条缝。

今天能用的三条：

1. 每周记一行“我顺手多做的事”：做了什么、谁因此少干了什么。
2. 动手前问一句“缺了会不会疼”，不疼就不做。
3. 做完留一句能让别人接手的话，别让它变成“只有你会”。

第 10 章 被源码按在地上摩擦的那个凌晨

一、拆一台没有说明书的洗衣机

读源码这件事，很像拆一台没见过牌子的洗衣机：外壳摸得到，说明书没有，螺丝还都是内六角。

第 5 章里，我第一次点开框架的源码，看了二十分钟，默默关上。那次不算读，算看了一眼就跑。

关上的原因可以有很多，但最不需要的一个是“我看不懂”。更常见的情况是：不知道从哪儿看起。这跟聪明不聪明没关系，跟入口有关系——一栋没有门的楼，谁进去都得爬窗。

被按在地上摩擦的第二次，发生在一个凌晨——读的是 jHipster 的源码。体感只有一句：屏幕上的字都认识，连起来一句都不懂。

还有一件事我后来承认：读源码最费的不是脑子，是耐心。得允许自己在同一个文件里绕三圈，还得忍住不去重构它——你是来找答案的，不是来当评审的。忍住了，答案会出来；忍不住，你就从一个排查问题的人，变成了一个给别人的代码写意见的人。

二、文档说应该，源码说实际

那一夜之后，我拿到的判断是这一句：

文档写的是“它应该怎么工作”，源码写的是“它实际怎么工作”。两边打架的时候，赢的永远是源码。

为什么文档会骗人？因为文档写的是作者的意图，源码写的是实现的历史。一个框架活了几年，里面很多行为不是设计出来的，是修补出来的——修的时候没人写文档，因为“这是个小事”。小事攒多了，文档和代码就分家了。

这也能解释一个常见困惑：为什么“官方文档明明这么写”，跑出来却不是。因为写文档的人和写代码的人，不一定是同一个人，也不一定在同一个季度。

再往下说一层，更扎人：

源码是程序员最后的技术文档，而且唯一一份永远是最新版的。

文档会落后一个版本，博客会落后一个框架，问答网站会落后一个公司。照着它们改，改出来的东西能编译，但行为不对——因为写那些字的人，和手上这份代码，不在同一个时间点上。

这话听着像“快去读源码”，我知道。所以我说小一点：**不是劝人去读源码，是让人在没别的路的时候，知道源码在哪儿。**

三、带着假设进，找到就撤

带着问题读源码，我只用三步，多一步不做。

第一步，带着假设进。

进去之前先写一句：“我怀疑是 X。”读源码只为证实或证伪这一句。没有假设的通读，等于在图书馆里散步——走得很远，什么也没带走。

第二步，只抓一条主线。

从调用的那个入口往里走，只跟一条调用链。遇到分支，记下位置，先别跟。先看控制流，再看状态：先弄清“它按什么顺序做了什么”，再弄清“它当时手里拿着什么”。

第三步，能解释现象就撤。

找到那个能解释现象的点，记下三样：哪个文件、哪一行、什么条件下会这样。然后停手。我当年不懂这个“停”字，非要读到全懂，结果是读了三天，懂了三天前的问题。

还有三个坑，我全踩过：

- **从第一个文件读起。** 源码不是小说，第一页通常是许可证声明和版权信息，读完只会获得一种“我开始了”的错觉。
- **所有分支都跟。** 一个方法里有五个分支，其中四个跟你的问题无关。全跟一遍，等于把别人的整个业务史复习了一次。
- **读完不记录。** 那一晚觉得“这辈子都忘不了”，第二天早上只剩下“好像是在某个 Manager 里”。

补一句实操：找到那一行，先写下来再高兴。记什么？文件、行号、触发条件、以及“如果这里改成这样，会发生什么”。四行字，够以后的自己少熬一个夜。

还有一个偷懒的办法，我后来常用：**先看测试。** 测试里写着“这个功能应该表现成什么样”，比读实现快得多。如果这个框架连测试都没有，那你至少知道了一件事：它为什么敢这么改。

四、读完，框架就不是魔法了

那一夜之后，我对框架的态度变了一件事：我不再把它当魔法。

魔法是没有解释的，源码有。

但我也得说句实话：读源码的收益不在那一晚，在之后每一次遇到同类问题时的“路径感”。你知道该往哪儿看，知道大概是谁在捣鬼，知道哪些地方不值得看。这种东西没法速成，只能一次一次攒。

也得说清楚什么时候不该读：改一个配置就能好的事、能问到一个肯定知道答案的人、以及只是想显得勤奋的时候。这三种情况读源码，收益都很低，还容易读出一肚子改造欲望——最后你解决了自己造出来的问题。

拆完一台机器，人就容易手痒：这玩意儿到底分几层？

本章的一句话：文档说的是应该，源码说的是实际；两边打架，源码赢。

今天能用的三条：

1. 打开源码前，先在纸上写一句“我怀疑是 X”。
2. 只跟一条调用链，遇到分支记位置、先不跟。
3. 找到能解释现象的那一行，记下文件、行号、触发条件，然后停手。

第 11 章 原来框架是分层的

一、不是不行，是叫错了名字

我原来看框架，像看一栋楼的外立面：只知道“能住”。

后来才知道，楼里有地下室、有承重墙、有装修层。在装修层上折腾，最多难看；在承重墙上开个洞，整栋楼跟着一起塌。

第一次“看见楼层”的瞬间，不是看懂了哪段代码，是改了一个自己以为只影响一处的小地方，结果楼上一层跟着晃了一下。

我这套东西后来长出了 Java 版、前端版、Python 版、.NET 版好几个分支。一个内核，能长出好几种外壳，这件事本身就说明——它里面是分层的。不然长不出来。

在看见分层之前，我见过的“框架”大多是同一个形状：一个 util 包，一个 Base 类，外加一堆 if。它能干活，而且干得挺快——问题在于，所有人（包括我）都管它叫框架，然后就按框架的标准去要求它：要通用、要扩展、要别人也能用。它做不到，于是大家得出一个结论：这东西不行。

其实不是不行，是**叫错了名字，用错了标准**。

二、别看代码行数，看谁调谁

这一章最想说的判断，很短：

判断一个东西是不是框架，别看它有多少行代码，看谁调谁。

工具类是“你调它”。写下 `DateUtil.format(now)`，它老老实实返回一个字符串，它不会自己跑起来，也不会在没人写代码的时候来找你。

框架是“它调你”。把一段逻辑放进它的扩展点，到了该执行的时候，它来调。甚至不知道它什么时候调的——这就是“控制反转”，大白话就是：**不是你调框架，是框架到时候来调你。**

再补两句，也是我这些年用来吵架的：

- 只有我一个人调它的，叫工具类；别人也在调它的，叫组件；它反过来调你的，才算框架的雏形。
- 没人维护的框架，不如没有框架。因为它比“没有”多了一份“看起来有人管”的错觉。

还有个更不客气的说法：**层级不是身份，是责任范围。** 越往上，改一次，影响的人越多。所以“我能不能改”这个问题，在低层是技术问题，在高层的问法是“改完谁来承担”。

层级错位有三种常见姿势，我全都见过，也都干过：

- **在 L0 上谈架构。** 一个两百行的工具类，非要给它加插件机制、加生命周期、加事件总线。加完之后，它还是只被三个地方调用，只是没人看得懂了。
- **在 L3 上写业务。** 框架里塞满某个业务的具体字段、具体状态、具体规则。第一次很快，第二次换业务就得复制一份框架。
- **在 L4 上喊口号。** 平台化的名字挂上去了，维护者还是那一个人，接入文档还是口口相传。它不叫平台，叫“一个人的大项目”。

这三句话还有个隐藏用途：帮人决定要不要接一个东西。有人拿着一个工具来让你“接入框架”，先问一句“谁调谁”——如果还是你调它，那叫依赖，不叫框架。依赖是可以换的，框架是要谈条件的。

三、一张表，把自己对号入座

L0 到 L4，我后来给自己做了一张一页纸的对号入座表：

层	名字	一句话判别
L0	工具类	删掉它只影响几处调用
L1	基础库 / 通用组件	两个以上业务方在用，用的人不用懂里面
L2	脚手架 / 工程模板	新项目从它创建，五分钟能跑起来
L3	业务框架	它调你，你只填扩展点

层	名字	一句话判别
L4	平台	多团队共享，有专门的人和接入机制

这张表不难背，难的是往自己身上套。因为大部分人（包括当年的我）会把 L0 的东西叫成框架——名字起得早，责任没跟上。

五层里最容易被跳过的是 L2。它不出彩：不通用、不酷、也写不进简历。但它是唯一一层能直接省人天的东西——新项目五分钟跑起来，等于每个项目都少了一个“搭环境”的下午。这种价值没人给你发奖，因为它省下来的时间，散在别人身上。

叫错层级有个现实代价：**你会用上一层的要求，去骂下一层的自己。** L0 的东西被要求“支持多业务”，于是你给它加配置、加钩子、加抽象，最后它变成了一个四不像——工具不像工具，框架不像框架，维护成本倒是很像个平台。

三个问题，拿去问手上那个“我自己写的框架”：删掉它会影响几处调用？有没有第二个业务方在用？它出问题的时候，除了作者还有谁知道去哪儿看？

三问里最狠的是第三问。前两问答错只是叫法不准，第三问答错，是给自己签了一份无期的值班表——因为前两问还能补救，第三问只能靠人。

四、像第一次看自己的 X 光片

那天之后我再看自己的代码，感觉像第一次看自己的 X 光片：原来一直在一楼装修，还以为自己在盖楼。

分层的另一个好处，是知道自己现在不该干什么。L1 阶段天天想着造 L4，是这行最常见的着急——因为 L4 听起来像成就，L1 听起来像打杂。

看懂了层级，手就开始痒。

我想改它。

本章的一句话：工具类是“你调它”，框架是“它调你”；分不清层级，就会在错误的楼层使劲。

今天能用的三条：

1. 数一遍：删掉它，会影响几处调用。

2. 列一遍：有没有第二个业务方在用。
3. 问一遍：出问题时除了作者，还有谁知道去哪儿看。

第 12 章 第一次改框架的代码

一、没人拦，不等于没问题

第一次改框架，很像给别人家的承重墙开洞：图纸不在手上，物业不在身边，还觉得自己挺懂结构。

但我要老实交代：我第一次改的框架，是我自己造的。

betterlife 这套东西，2010 年开始写，那时还只是个雏形。所以严格说，第一次给框架动手术，动的不是别人家的墙——是自己家。

自己家砸墙有个好处：随便。也有个坏处，更实在：**没人拦。**

更麻烦的是，“没人拦”很容易被误认成“没问题”。改完跑得通、上线没出事，人就会得出一个结论：改框架这事，我挺在行。前半句是真的，后半句是运气。

那次是被什么逼的、改了哪几行——当时觉得改得挺顺，事后一点都对不上号；怎么验证它是安全的、上线那晚什么状态——也是全凭一句“跑得通、没出事”带过去的。

能确认的只有一句，用我自己的话：“当时有想法要改，就去改了。”

这句话现在看，像病历，不像经验。

二、能低不高，越深欠账越多

改框架这件事，我后来把它拆成四层，从低到高：

配置 → 扩展点 → 覆写 → fork。

规矩是四个字：**能低不高。**

为什么？因为每往下一层，欠的账就翻一倍。

- **配置**：只改一个值。升级的时候，这个值还在。代价最低，因为它没碰逻辑。
- **扩展点**：只填一段逻辑。框架留了钩子，挂上去，它负责调。代价是你要按它的规矩写，不能随心所欲。

- **覆写**：把它的一个方法换成自己的。听着很轻，其实已经开始跟它的默认行为分家了——以后它改了默认值，你不知道。四层里最难判断的就是这一层，因为它藏在代码里：评审的人看到的是“多了一个新方法”，看不到“一个默认行为被悄悄替换了”。
- **fork**：把整个东西搬回家。从此以后，上游每改一次，都得自己再改一次，而且不知道它改了。它不是改代码，是接手一个项目。

我当年最熟的是后两层。原因不复杂：前两层需要框架先留口子，后两层只需要胆子大。

这就是“有想法就去改”的代价：它省了当时的决策时间，把账推到了之后的每一年。而且这笔账不会自己出现，它会藏在某次升级、某个新人接手、某个深夜排查里。

还有一层心理原因，得说破：**越往深改，越有掌控感。** 配个参数，五分钟，没人记得；改核心代码，三天，你会觉得自己参与了这件东西的建造。人很容易把“参与感”当成“价值”，把“改得深”当成“懂得深”。

三、动刀前，先留基线和退路

现在每次有人问我“这个需求该不该改框架”，我只让他回答四个问题：

一、配置能不能解决？

能，就别动代码。九成的“框架不支持”，其实是“我没找到那个配置项”。剩下的一成，是文档没写清楚。

二、它有没有留扩展点？

有就填，没有就问：这个扩展点该不该有？如果该有，那要做的不是改，是提一个“能不能留个口子”。

三、是要覆写一个方法，还是复制一整段？

覆写只换一个点。复制一整段，其实就是没走流程的 fork，换的是一整条命——包括它以后所有的 bug。

四、真要 fork，先答：上游升级，谁来跟？

答不上来，就别 fork。那不叫 fork，那叫分手，而且分得不体面。

这四问里，没有“直接改核心代码”这一项。忍不住改了，那就是 fork，只是没办手续——手续没办，账一样要还。

动刀之前，还有三个动作，比四问更救命的：

1. **留基线。** 改之前先记下：现在的表现是多少、正常范围是多少。没有基线，改完你都不知道自己是变好了还是变坏了，只能靠“感觉没出事”。
2. **留退路。** 一个开关、一个旧版本、一句“一分钟能退回去”。改框架最怕的不是改错，是改错之后退不回去。
3. **留一句话。** 为什么改、改了哪儿、什么情况下会不一样。写在提交信息里，给三个月后的自己——那个自己一定不记得今天。

还有一件小事，我是吃了亏才记住的：**改完框架要告诉用它的人。** 不用发公告，在群里说一句“这个行为变了”就够。不说，别人会以为是自己写错了，然后花一个晚上排查一个根本不存在的 bug——那个晚上，本来该算在你账上。

四、“只有我改得动”，是句判词

改得越多，手越顺。顺到某一天，脑子里冒出一句很舒服的话：

“这地方只有我改得动。”

这句话之所以舒服，是因为它同时满足了三件事：我懂、别人不懂、这件事离不开我。

我当时把这句话当夸奖收下了，甚至觉得，这是“深度参与”的证据。

很多年以后才知道，那是一句判词。不过那是第 16 章的事，现在先让我飘一会儿。

更准确地说，那句话只是判词的开头。判决要很多年后才念完：当所有改动都只能由一个人完成的时候，框架就不再是作品，而是一张排班表——上面只有一个人的名字，全年无休。

手痒之后，我开始攒自己的东西——攒到第三次写同一段代码的时候，我停手了。

本章的一句话：改框架有四个层级，能低不高；每往下一层，欠的账翻一倍。

今天能用的三条：

1. 遇到“框架不支持”，先把配置项翻一遍，再谈改代码。
2. 想让框架留口子，就提“扩展点”，而不是先改源码。
3. 想 fork 之前，先写出“上游升级谁跟”这一行；写不出，就别 fork。

第 13 章 第三次写同一段代码时，我停手了

一、复制粘贴是止痛药

复制粘贴是程序员的止痛药：见效快，不治病，还容易上瘾。

疼的地方很固定：报表要导出、列表要分页、表单要校验、增删改查要写第五遍。同一段代码，第一次写的时候觉得是业务，第二次写的时候觉得是手感，第三次写的时候，开始觉得是命。

重复有三种形态，长得都不太一样：

- **复制粘贴**：代码一模一样，只是文件名不同。
- **口头流程**：每次都有人问“这个要怎么发”“这个表要怎么建”，答案永远在某个人的脑子里。
- **手工操作**：每次发版都要手动改三个配置、导一次数据、点五次按钮。

第一种伤代码，后两种伤时间。而且后两种更隐蔽——因为它们不会被代码扫描工具发现，只会被发现“这个人怎么这么忙”。

还有一种重复，比这三种都难看见：**同一个问题，每次换一种办法解决。** 今天写脚本，明天手工，后天找同事。它不产生第二份代码，它产生的是一个总在救火的人。

那种感觉说得清：手指敲的是第三遍，脑子里想的是“我是不是在给自己造一个以后要维护三次的麻烦”。

二、抽得太早，比重复更贵

这一章想说的判断，就这么一句：

组件不是设计出来的，是被重复逼出来的。

不是“我觉得这里该抽象”，而是“这段我写第三遍了，再写一遍我会恨自己”。前者是审美，后者是疼。审美会变，疼不会。

但我还得补一半，因为只说前半句会害人：

抽得太早，比重复更贵。

只重复过两次的东西，别急着抽。还没看清它的变化点在哪儿，抽出来的东西会变成一个更贵的复制粘贴：多了一层包装，多了一个没人敢改的文件，出事的时候还多了一跳要排查。

抽象过度长什么样？一个方法八个参数，其中六个永远是默认值；一个组件带三层配置，最后一层谁也不敢动。它看起来比复制粘贴高级，实际上是把三份重复变成了一份看不懂。

所以那条土办法一直留着：**重复第三次，停手。** 三次足够让人看见“哪部分在变、哪部分不变”——这两样看不见，抽出来的东西一定错。

（顺便说个真话：我这套框架的代码生成，也不是先设计好再写出来的。它是一堆“我这几天懒得写哪几段”攒起来、再一条条改规则改出来的。先有重复，后有规则，顺序是反的。）

三、四条缺一条，就先别抽

真要抽，我只有一个最低标准清单，四条，缺一条就先别抽。

一、单一职责。

一句话说清它干什么。说不清，就是两个东西粘在一起了，只是暂时住在一个文件里。检验方法很土：给它起名字，名字里出现“And”或者“以及”，就该拆。

二、有配置。

至少留一个参数。把所有业务写死的东西，别人拿去只能照着改，那就没抽。参数也别多，多到没人记得住，等于没有。

三、有日志。

出问题的时候，要能知道是谁调的、带什么参数进来的。没有日志的组件，出事时只能靠猜——而猜的人通常就是作者。日志不用漂亮，一行就够：谁、什么时候、带了什么。

四、有失败处理。

失败的时候给人留退路：能重试、能降级、能关掉。最省事也最缺德的做法是吞异常——吞下去，问题就归下一个人了。

还要加一条“不抽”的判断：只在一个地方用、半年内也不会变的东西，就让它重复着。它不是债，它是还没看清的一段业务。

抽完之后，还有一步最容易被跳过：**写一段示例调用。** 不用长，五行就够——别人看这五行，就知道怎么接、传什么、拿到什么。没有示例的组件，接入成本全靠问作者；问得多了，作者就成了文档。

反过来也有一个判断：如果抽出来之后，接入的人还得问你三遍，那说明抽出来的不是组件，是自己的习惯。

四、一个人用得顺，不算活着

那个东西写完之后，就躺在代码里。有没有别人在用，我当时并不知道。

这不算成功，只能算没浪费。

说句实话，抽组件最大的收益，通常不是省代码，是省脑子：不用每次都回想“这段以前是怎么写的”。省下来的那点注意力，才是能拿去干别的事的东西。

一个人用得顺，不等于它活着。什么时候才算数？

下回说。

本章的一句话：组件是被重复逼出来的，不是被设计出来的；重复三次再停手。

今天能用的三条：

1. 数一遍最常复制的那段代码，出现了几次。
2. 够三次的，写下“哪部分在变、哪部分不变”；写不出来就先不抽。
3. 抽的时候四条过一遍：单一职责、有配置、有日志、有失败处理。

第 14 章 第一个被同事复用的东西

一、好用到只有我一个人用

我写过不少“特别好用”的东西。好用到什么程度呢——只有我一个人用。

这就像家里腌的那坛咸菜。腌的时候你很有把握：盐量准，坛子擦得干净，闻着就香。封好放进柜子，一年到头自己夹。偶尔有人问一句“这是什么”，你说“我自己腌的”，对方“哦”一声就走了，你还会替他惋惜：他不懂。

我最早那批工具类就是这么一坛咸菜。写的时候很爽，心里想的是“谁用谁知道”；写完往项目里一放，确实只有我知道。

那时候我对“好用”的定义很省事：我自己用着顺手。这个标准永远不会失败，因为它只考核一个人。

二、夸是礼貌，用是投票

第一次真的有人把我写的东西拿去用，是一次赶工。

那是 2012 年，我在卡司特曼——我和同学合伙开的公司。项目要开张，从零起，时间只剩两周。我用 C# 搭了一套快速开发的底座，交给开发团队用，项目按期交付。这件事我后来记错了好几年，直到按最早简历核对才改回来——记忆不比提交记录可靠。

交付那天我挺得意，觉得自己干了件漂亮活。现实随后就来收利息：交付只是开始；有人不问我、直接拿它去干活，才是真正的开始。

第一个拿去用的，不是我找的人。那东西是一套模板，我没主动递给他，是他自己发现的；发现之后，就开始用了。至于他说了什么、后来我为它改了什么，这段我确实记不全，也不替他补一句台词。

模板和工具类不太一样：工具类是别人来借你的锤子，用完还你；模板是别人照着你的样子，另起一间屋子。借的人多了，锤子还是那把；照的人多了，你才知道自己那套东西里，哪些是真规矩，哪些只是你的习惯。

但我记得那个感觉：有人用，和有人夸，是两件完全不同的事。夸是礼貌，用是投票。

所以我这章想说的判断是这一句——

别人用不用，是组件唯一的及格线。

还有一层是我当时没料到的：被复用不是被表扬，是被提要求。用的人越多，“这个默认值不对”“这个报错看不懂”“能不能加个开关”就越多。这不是麻烦，这恰恰说明它真的在工作。一件从来没人提要求的东西，通常也没人在用。

三、接不住的，长在作者身上

后来我琢磨：为什么有的东西别人接得住，有的接不住？

答案跟“写得好不好”关系不大，跟“接的时候要不要先问你”关系很大。

我早年那些没人用的东西，毛病都一个样：默认值是我脑子里的默认值，参数一个不能少；接入方式只有我会；出了问题，日志只在我脑子里。那不叫组件，那叫我的私人器官——长在我身上，别人借不走。

我后来给自己定了一个很土的标准：一个东西能不能算组件，不看它写得多巧，看它能不能被一个没见过它的人接住。接住的意思是，他不用读源码、不用问你，照着说明调一遍就能跑；跑不通的时候，他知道该去哪儿看。

这个标准一立，我那些“特别好用”的东西，一大半当场就不合格了。它们不是不好，是只对我好。

那三条要求都不高级：不填也能跑、接入看得懂、出了问题查得到。但它们决定了一个东西到底是“我的”，还是“我们的”。

这也解释了我早年一个可笑的判断：我把“别人没在用”理解成“别人没眼光”。真实情况往往相反——别人试过一次，发现得先问你三个问题，就再也没试第二次。

模板尤其如此。它连“来问你”这一步都省了：看着像，抄走就能用；抄走之后，它就已经不完全属于你了。那是我第一次尝到这种滋味——东西还在我手上，用法却已经长到别人身上。

四、默认值、一页纸、一条能查的路

所以这一章能带走的，不是“多写组件”，是三条很具体的接线活。

一、默认值。 能不传的参数就不传；非要传的，给一个能跑起来的默认值。检验方法很土：找个没接触过它的人，只给他一页说明，看他能不能跑通。他跑不通，不是他笨，是默认值在替作者说话。

二、一页纸。 只写三行：它是干什么的、怎么调、出问题先看哪儿。超过这三行，多半是它自己还没想清楚。别写“架构说明”，那玩意儿只有作者看得懂，也只有作者爱看。

三、一条能查的路。 日志、错误码、一个能复现的最小例子，三样里有一样就行。这一条最容易被省，也最容易被报复：省下的是一行日志，还回去的是三个人的半天。

那次被复用，让我飘了很久。我想造个大的，从组件到框架。

在我当时的理解里，这是从“我帮大家省事”走到“大家离不开我”——听起来像升级，其实就是换了一种负债方式。

那年我以为自己终于有了作品。后来才知道，作品和负债，在账面上长得一模一样。

本章的一句话：别人用不用，是组件唯一的及格线。

今天能用的三条：

1. 给你手上最“好用”的那个工具类，补一个能跑的默认值。
2. 给它写一页接入说明，只写“干什么、怎么调、出错看哪儿”三行。
3. 找一个没碰过它的人照着念一遍，卡在哪儿就改哪儿。

第五篇　造与交

第 15 章 我造了个框架，然后只有我用得顺

一、修水管的，只有漏水时才被想起

官网要改版。

这种事在公司里排不进任何一份周报的标题。它不算项目，算杂活：没有立项会，没有排期表，没有验收会，做完也不会有人开香槟。

官网我前后改过好几版。最后一版之前的几版，都是我用 betterlife 搭的；最后一版我只留了一部分，剩下的交给做前端的那位——但真正用 betterlife 搭系统的人，从头到尾只有我一个。能确认的是那种手感：模板是现成的，栏目拖一拖、样式调一调，一个下午能出来别人两周的活。

上线之后，群里有人说“看着还行”。然后就没了。

我关了电脑，觉得自己像修水管的：家里不漏水了，没人会感谢水管工；只有漏的时候，所有人都会想起他。

过了很久，官网出了个小毛病。第一个被叫的还是我。那一刻我才回过味来：它平时没人提，坏了却第一个被想起。

二、不重要的系统，在偷偷省时间

这话不是我抬举自己。用 betterlife 那几年，我做的东西一直不太“上台面”：好几版官网、帮助系统、报表系统，还有 redis helper 的前身。按我自己的说法，这些是核心产品的辅助系统——大意是：“虽然不重要，但是却大大地减少了公司的工作量。”

不重要的系统，往往在偷偷替你省时间。

“不重要”这三个字很实在：它不背 KPI，不进汇报 PPT，年终总结里也占不了一行。可它一坏，客服答不出问题，运营拿不到数，官网挂在那儿像关了门。省下来的时间从来不在账上，因为它分散在每个人每一天的十分钟里。

这里还有一层，我当时没想明白：这些系统都是用 betterlife 搭的。它从 2010 年那个雏形起步，到 2015 年才成熟；从那时候到现在，用的人只剩我一个，没有

任何其他人。原因不复杂——我是它的原作者，知道怎么把它用到最顺，也只有我能往上拓展开发。别人不是不能用，是要先学会我的用法。

别人不用，不等于它没用；但“只有我用得顺”，是优势，也是陷阱的入口。

优势是：同样的活，我干得比谁都快。陷阱是：快到最后，所有活都只剩我能干。

三、没人催着做，一坏全卡住

怎么认出这类“不重要但很值钱”的系统？我后来只用三条：

一、没人催着做，但它一坏所有人都卡住。

没人立项，说明它不出彩；一坏全卡，说明它在承重。

二、每个项目都要重复一遍、又没人愿意重复的那段活。

建表、导数据、配环境、写导出——谁都嫌烦，谁都还得干。这种地方最值得做一次。

三、做出来不出彩，但做一次能省很久。

它的收益不体现在上线那天，体现在之后每一次“这次不用再弄了”。省下的是重复，不是惊喜。

认出来之后，还要补一件最不性感的事：**写一页纸的接入说明。** 干什么、怎么接、出错先看哪儿，三行。没有这三行，再好的东西也只能长在作者身上。

说到这儿，得把两件事分开：**造框架和用框架做事，是两件事。** 前者我是作者，产出的是一套规则；后者我只是用户。两个身份叠在一个人身上，才显得格外顺。

辅助系统说的是“我用它做了什么”；接下来该说“它自己长出了什么”。redis helper 和 bb.report 没有先后，是从 betterlife 演变出来的两条分支——都同源，不是父子。redis helper 干的事很窄：可视化地查、改 Redis 的 key 和 value。从 2015 年到现在，我在做 Java 后端、又顺手折腾 Redis 可视化工具的圈子里，没见过比它更顺手的。这话只限这个圈子；你要是见过更早、更全的，欢迎把仓库地址、发布时间和功能清单摆出来——我接得住。

岔开一句：Redis 是一个人写出来的

redis helper 是围着 Redis 做的，顺带说一句它是怎么来的。

Redis 是 2009 年由 Salvatore Sanfilippo 用 C 写出来的。之后的十一年，主要维护者一直是他——那十一年里，维护者那一栏没有第二个名字。2020 年，他公开宣布退到二线，团队在官方博客上发文谢过他；过了几年，他又回来了。

你手机里那些 App 的临时数据，很多就存在这类地方——它把数据放在内存里，专管“现在这一刻”的数字，所以快。

一个人、一个项目、十一年。这三样摆在一起，比任何架构图都说明一件事：一个东西能活下来，靠的往往不是设计得多漂亮，是有人一直在。

这些我当年都不知道。我只知道它快、好用，然后就围着它做了个工具。

后来 AI 那阵，我用 AI 把它又做了一遍。界面和功能基本一致，补的都是我自己用着别扭的几样：key 排序、回收站、批量删除、首页统计、性能评估。底座换了——Rust 写后端，Tauri 做壳，Web 那一版还在，前端是 Vue.js，另外用了 WebSocket。换底座和加功能，是两件事；把它们讲成一件“觉悟”，是我事后给它加的戏。

还有一个旁支得交代：betterlife.front。它是独立的前端框架，给 betterlife 供前端底座和通用模板。现在没用了。我留着它，不是因为舍不得，是因为我还没想清楚它该不该活着。

四、钥匙和锁，都交到自己手上

用得越顺，我越不需要别人；越不需要别人，这套东西就越只剩下我一个人能碰。

当时我以为这叫各司其职：我用我的，我改我的。很多年以后才看清，那不是分工，是我把钥匙和锁一起交到了自己手上。

顺手这件事还有个副作用：它让“别人上手”变得没必要。我十分钟就改完了，别人花半天也未必改得动；既然有我，谁还会去试？没人试，就长不出第二个人；没有第二个人，这套东西的寿命就只等于一个人的工龄。

省下来的时间是实的，欠下的维护也是实的。账不在一开始收，在后面每一年慢慢收。

那时候我还挺得意：框架成熟了，我也成熟了。直到有一天发现，成熟的东西有个共同点——离了某个人，就转不动。

本章的一句话：不重要的系统，往往在偷偷替公司省时间；而“只有我用得顺”，是它开始向你收账的信号。

今天能用的三条：

1. 列出三件“没人催、但一坏全卡”的东西，看它们现在谁在维护。
2. 给你最顺手的那个内部工具，补一页纸：干什么、怎么接、出错看哪儿。
3. 找一个没碰过它的人，只给这一页纸，看他能不能跑通——卡在哪儿，就改哪儿。

第 16 章 只有我能改

一、家里只有一把钥匙

家里要是只有一把钥匙，会出什么事？

平时没事。钥匙挂在你裤腰上，你开门、你锁门，一家人进进出出都得等你。你觉得这叫方便：钥匙在谁手里，门就听谁的。

直到有一天你出差、生病、手机没电，或者只是睡着了。门口站着一家人，谁也没带钥匙。

我给 betterlife 装的就是这么一把锁。原话是：“完全只有我自己能改。”

产品后期的用户改善需求，看代码、改代码、测一遍、发出去，全是我。具体是哪一次需求、谁提的、来回折腾了多久，早记不清了；记得清的是那种绕法——提出来，先转一圈，最后还是落回我这里。

那时候我不觉得这有什么问题。甚至有点受用：你看，这事还得我来。

受用是有原因的：在一个团队里，“只有你能搞定”是最容易换来的安全感。它不像加薪那样要谈，不像晋升那样要评，它只要你多干一点，就自动到账。所以人很容易上瘾——包括我。

上瘾的表现不是加班，是舍不得。别人伸手的时候，你会下意识地想：他改坏了怎么办？他要问我一堆问题，还不如我自己来。这个念头每出现一次，那把锁就再焊紧一点。

二、不是能力证明，是设计事故

现在我把它说清楚：**“只有我能改”不是能力证明，是设计事故。**

它证明的不是我强，是这套东西只兼容我一个人。一个系统如果只有一个人能改，那它的寿命就等于这个人的在岗时间；这个人生病、休假、换工作，系统就跟着停。

更要命的是，这通常不是别人不肯改。我自己的原话把原因摆得很明白：当时对 GitHub 不了解，本来可以让其他开发者加入，但自己不重视开源协作的理念，只把它当作“个人锻炼的一个架构”；“当时有想法要改，就去改了”。

翻译成人话：门一直开着，但门口没写“欢迎进来”，也没放鞋。路过的人看一眼，就走了。

这套东西在别人眼里还有个特点：神秘。会写的人觉得它省事，不会写的人觉得它高深。高深很容易被误读成价值，价值又很容易被误读成“还是让他来吧”。于是每一次“还是让他来吧”，都在给那把锁再加一道栓。

第 12 章那句“这地方只有我改得动”，我收下的时候是当夸奖的。夸奖和判词的区别，在于有没有人跟着受益：很多人受益，那是夸奖；只有我一个人受益，那是判词。

不开 issue、不收 PR、文档不全、把项目当个人练习——这四件事凑在一起，就等于给自己焊了一把独一无二的钥匙。

而且“有想法就去改”这件事本身，也是成本。它省了当下的决策时间：不用讨论、不用评审、不用告诉谁。账没有消失，只是被推到了之后每一年——推到某次升级、某个新人接手、某个深夜排查里。

三、先问有没有第二个人

后来我用三个问题判断一个东西该不该推给更多人用，缺一条就先别推：

一、扩展点够不够用？

别人要加一个自己的需求，是改配置、挂扩展点，还是必须动核心代码？如果答案永远是后者，那就是没留口子。

二、文档能不能让第二个人上手？

不是“我讲过一遍”，是“我不在，他照着能跑”。检验方法很土：找个人，把文档给他，你闭嘴。

三、出了事有没有第二个维护者？

不是“他愿不愿意帮”，是“他有没有这个权限、这份上下文”。没有，就是没有。

这三条都不高级，但它们决定了一件事：这东西是作品，还是排班表。排班表上只有一个名字的时候，那个人是没有假期的。

留入口不需要什么大动作。一个 README、一页接入说明、一个愿意回的 issue，就够了。入口不是门面，是让别人敢迈第一步的那块踏板。踏板没有，人不是不想进，是不知道往哪儿踩。

四、“这次先我来”，说了一百次

那几年我一直觉得，这只是暂时的忙。等版本发完、等需求做完、等这阵过去，我就有空把口子留出来。

“这阵”一直没过去。

需求一个接一个，框架一处一处改，产品一行一行往上长。我一边完善框架，一边开发产品，两头都要我，两头都只有我。

我不是没想过放手，想过很多次。每次想到一半就停了——不是舍不得，是算不过来：教一个人看懂这套东西要花很久，我自己改一会儿就完。“这次先我来”，说一次没什么；说上一百次，它就成了制度。

我以为人会一直撑得住。直到身体先替我做了决定。

本章的一句话：“只有我能改”不是能力证明，是设计事故——多半不是别人不肯改，是你从没给别人留入口。

今天能用的三条：

1. 挑一个“只有你能改”的东西，试着让别人改一个小地方，你在旁边看，不动手。
2. 给它补一页“我不在时怎么办”：找谁、看哪、先做什么。
3. 下次想“有想法就去改”，先写一行：这个想法以后谁来维护。

第 17 章 我把前端推倒重写

一、哪儿坏了修哪儿，修到没处住

老房子住久了，会养成一种错觉：哪儿坏了修哪儿，房子就还能住。

今天补一块墙皮，明天换一根水管，后天把门轴上了油。每次修完都挺有成就感——你看，又省了一笔装修钱。

直到某天你发现，修的地方比住的地方还多。问题不是这块砖那块砖，是最开始选的那套骨架，跟现在要住的人对不上。

Ext JS 就是那套骨架。

betterlife 第一版的前端用的是它。当年选它不算错：组件齐全，界面规整，配合代码生成，很多重复的页面工作确实省掉了。省掉到什么程度？省到我一度觉得，写业务这件事已经被我解决了一半。

但产品开发到了后期，问题换了个方向：用户提改善需求，要改代码。而这类改动，经常必须我亲自动手。具体是哪一类需求、哪一次最典型，已经说不上来；只记得它们都不大，是那种在别处顺手就能改、在这套前端里得先找半天的改动。

一开始我以为只是这一段特殊。后来发现不是：只要动到前端，路最后都会拐到我这儿。

二、成本不在造它，在用它

所以这一章想说的判断是：

框架的成本不在造它的时候，在它被用起来之后的每一年。

造的时候是资产：几周、几个月，看得见东西长出来，很有成就感。用起来之后是负债：每一次需求变化，都要有人去翻那套只有他熟悉的结构。

技术栈的选择也一样。它从来不是时髦问题——时髦会过期，账不会。真正的判据只有一条：**这门技术，把改动成本压在了谁身上？**

Ext JS 把账压在了我一个人身上。它必须被换掉，不是因为它过时，是因为这笔账没法再往下记了。

当年选它的时候，我算的是“它现在能给我什么”：组件齐全、上手快、能出活。我没算的是“它以后会把改动压给谁”。两个问题只差几个字，答案差好几年。

三、两天换完，一年多才撑起来

后来我确实换了。但“换”这件事，得拆成三件，混着写就容易变成自我美化：

第一件：设计负担。 这是 Ext JS 时期的事，跨度很长，和仓库动作无关。产品后期的改动必须我本人动手，我一边完善框架，一边开发产品。人被两头拉着，哪头都松不了手。

第二件：仓库里的切换动作。 这部分有提交为证：2016 年 11 月 9 日 betterlife 主仓建立；11 月 16 日，bootstrap 前端随仓库落地；11 月 17 日，Ext JS 被删除。把新的放进来、把旧的删掉，前后两天，动作就做完了。

两天，听起来很利索。但这两天只是“换完”的开头——把一个前端另起炉灶，和把一个前端真正撑起来，是两件事。

第三件：换完之后的长期补齐。 仓库里那个 bootstrap 目录，全史六十多个提交；其中 2017 年 11 月到 2018 年 1 月是最密的一段（这是我的推断，依据是提交分布）。我记的是“花了不少时间”——这一句比任何数字都准。

换技术栈真正花时间的，不是“换”，是换完之后那一年多里，一点一点把它撑起来。 仓库能证明你哪天动了手，证明不了你熬了多久。

还有一件和这些技术内容无关的事：那段时间，我的身心出了问题，一直到现在。我不打算在这里解释它为什么发生，也不打算把它写成一段“挺过来了”的故事。它只是我算这笔账时，唯一算不明白的一栏。

四、造之前，先过三道闸

换完之后，我给自己立了三道闸。任何“我想造个东西”的念头，先过这三道：

一、谁在调它，有几个人在调？ 只有我一个人调，它就是个工具类；别人也在调，才算组件。

这套东西最热闹的时候，是还在卡司特曼那几年，公司里七八成的人都在用，最多的时候七个人。现在只剩我一个。别的公司还有没有老同事在用，我不知道，也没去查。

二、改的人有几个？ 只有一个，那它迟早变成第 16 章那把钥匙。

三、出问题谁来兜？ 答“我来”之前，先想想你能兜几年。

代码生成也一样。它确实省了我大量重复劳动，但有三条边界，我后来才认清：

一、生成省的是手，不是责任。 生成出来的东西越多，要改的时候还是那一个人去改。

二、生成的代码要能被人读懂。 读不懂的生成物，等于把黑盒又套了一层。

三、生成器本身要有维护者。 规则改一次，所有生成物都跟着变——这件事得有个人管，而且不能永远是同一个人。

五、我一直在优化“造得更快”

推倒重写那天，我以为自己在做一次技术升级。

很多年后才承认：我一直在优化“造得更快”，从没想过“少造一点”。

造得快，是手艺；少造，是判断。手艺我练了很多年，判断才刚刚开始。

判断没有证书，也没有验收会——你只能在某次翻车之后，才知道它来过。而它来的时候，通常你正忙着别的事。

本章的一句话：框架的成本不在造它的时候，在它被用起来之后的每一年；技术栈选的是“把改动成本压在谁身上”。

今天能用的三条：

1. 给手上那套“只有你能改”的东西，写下三个数：用的人、改的人、出事兜底的人。
2. 翻一遍你最熟的那个框架的升级记录，看最近一次“改完之后又补了多久”。
3. 下次想加一层代码生成之前，先问一句：生成器坏了，谁来修？

第 18 章 一起把活干完：让东西离开我

一、官网交出去了，报表没走

交接那天，我把官网前端交了出去。

接手的，是团队里本来就写前端的同事。不是我把一个没人干过的活硬塞给他，是这活本来就该由他这类人干。

结果很清楚：官网前端交出去了。而报表和帮助，一直由我一人管理、开发。

同样是我用 betterlife 做出来的东西，一部分走了，一部分没走。这个差别，比我当时以为的更值得琢磨。

二、成熟的标志，是不再需要你

这一章想说的判断是：

真正的成熟不是造得更多，而是让某一部分从此不再需要你。

为什么官网前端能交？三个条件，缺一条都交不出去。

职责边界清楚。 “前端”本来就是团队里有人在做的活，不是我发明出来的一个岗位。它不需要有人先学会“我的那套思路”，才谈得上接手。

有对口的人。 接的人本来就在写前端。他接的是活，不是我的世界观。

技术栈通用。 HTML5、bootstrap 那一套，出了门还有人会，网上还有答案。不是只有我看得懂的独门手艺。

报表和帮助为什么交不出去？不是没人愿意接。是它们长在我一个人的上下文里：那张报表为什么这么分组、那个帮助页面为什么这么排序，答案在我脑子里，不在代码里。别人接手，第一天要接管的不是项目，是我的脑子。

还有一句容易误会的话：**“交出去”不等于“写文档”。**

写一万字文档，不如把活转给对的人。文档是必要条件，不是成功的标志。真正成功的放手是，活到了他手上，他来问你的时候问的是业务，不是你代码里的那个开关在哪。

三、不分彼此，也分不清边界

说到“一起把活干完”，还得补一段人。

被带过这件事，我印象最深的不是工作，是游戏。有一家公司，先做手机上的应用服务，后来转做 WAP 和手机游戏。我进去的时候对游戏有偏见，不太开心。那阵子公司进来一批很优秀的同行，下班一起打 Dota；有人手把手教我。我手操和意识都差，大家也玩得开心。这件事把我对游戏的偏见改掉了一大半——后来我在雪橇待得住，和这一段有关。

那几年我带过新人，也被带过。做天地传奇那个产品的时候，带过两个刚入职的 Java 程序员；后来在雪橇科技，团队十人左右，前后把五名初级开发带成了骨干，也在团队里建起了 Code Review。这些事在简历上会被写成“培养骨干”，看着像功劳；我自己更愿意把它看成“把活分下去”的练习。我的原话是：“大家在一起像兄弟一样，又何必分彼此。”

也正因为这样，那些“不重要”的东西——帮助系统、报表系统——一直没离开过我。大意是“虽然不重要，但大大减少公司工作量”。它们省下的是别人的时间，占住的是我的位置。

再往后，雪橇是这么来的。朋友的朋友，饭桌上认识。他做前端、做嵌入式 App 很强，后端偏弱。我脱口而出一句“有什么需要帮忙的叫我，没问题”，他当时眼前一亮。后来他去开公司，让我过去，给的是公司里最高的工资——我觉得有诚意，就去了。到现在也没离开过。中间公司有过几次波澜，险些放弃，最后都靠信任撑下来。

“不分彼此”听着很暖，代价也很实在：没有明确的师徒边界，也没有沉淀出一套可复制的传帮带机制。这一条我只陈述，不下结论——当时最要紧的事，是把活干完。

现在回头看，“不分彼此”其实有个副作用：它让协作变得很顺，也让边界变得很糊。活是谁的、谁该负责、出了事找谁，全靠默契。默契在的时候，团队像一台上了油的机器；默契不在的时候，连“这活本来是谁的”都要吵半天。

那几年，很多活都压在我一个人身上，我差不多是技术主管。有一天，我突然病倒。我给公司打了招呼，可心里冒出一个念头：这阵子正需要人，他们会不

会以为我在要挟？这个念头让我很郁闷。生病是真的，什么时候能回来却说不准。往好处想，这是病；往坏处想，它看着像拿身体讲条件——那阵子我最怕的就是这个。后来日子一天天过去，大家理解了我的情况，把压在我身上的活拿走一些，尽量让我做擅长的事、力所能及的事。

岔开一句：一幅画像、一张笑脸和几部剧

有一幅画像，到现在还在我脑子里：上帝坐在高高在上的座椅上，一动不动，头顶是教堂的屋顶，看上去非常高。我不知道当初是在梦里，还是在现实里的某个地方看见的。

我当时的第一个反应是：上帝爷爷这样，真不容易，我可不可以帮帮他？可他是上帝，我能做什么呢？我能想到的，就是把我的快乐带给他。

这天下，大概也只有我这么一个敢跟上帝开玩笑的小朋友。

我说：我有的是办法让你笑。神说：你只管逗我笑，我只管不笑。

我知道上帝是严肃的、神圣的，他不会笑。但我还是照着一日一工、循序渐进的法子来。直到有一天，我觉得时候到了，于是感觉到了他发自内心的笑。

那一刻，我无比的光荣、荣耀、开心、愉悦。

是爱让我认识了主。是爱让我灵感无限，是爱让我笑个不停，段子不断。

回头再看，《琅琊榜》比印象里更好看，《冰雪奇缘》也是，《破晓东方》也不错。

我向往、我渴望、我怀念的，是我父亲年轻时候的那张脸——阳光、豁达、成熟，有魅力。

《许我耀眼》里男女主人公的撕扯，我看得很过瘾，也让我对爱情的领悟又进了一步。《伟大的转折》则让我对团队协作沟通、对会议的重要性，以及对团队分工不分权的理念，有了更深的认识。

扯远了，回来说活。

四、交出去之前，先写四行

交出去这件事，后来我做成了一张四行的纸。有东西要从我手上走，就先写这四行：

- **边界**：这块活到哪儿为止，什么不归它管。
- **人**：接的人是谁，他现在在做什么，为什么是他。
- **决策**：当初为什么这么设计，哪些地方看着奇怪但不能改。
- **止损**：我离开一个月，它会不会停摆？会停在哪一步？

第三行最容易被省。它是“设计决策记录”，不是“操作手册”——手册教人怎么点，决策记录告诉人为什么不能乱点。省掉第三行，接手的人迟早会把一个“看着多余”的东西删掉，然后在一个月后出事。出事的时候他不会觉得自己错，他只会觉得那东西本来就没用。

第四行最容易被答得太快。很多人一句“不会停”就过去了，其实心里想的是“反正我还在”。真正的问法是：我不在，它靠谁转？那个“谁”现在知不知道自己是“谁”？

五、学会一半，也是学

也说一句冷的：一套系统如果要靠一个人的健康来维持，它在设计上就是失败的。而且这通常不是“没人愿意接”，是“从来没留过入口”。

交出去一部分之后，我手上还是剩了不少：报表、帮助、框架本身。

我不打算把这段写成“我终于学会了放手”。学会一半也是学，另一半还攥着。攥着的那部分，不是我不肯放，是我还没把它整理成别人能读懂的样子。

只是从那时候起，我开始问另一个问题：所谓合格，到底算不算合格？

本章的一句话：能不能交出去，取决于那部分知识能不能被别人读懂。

今天能用的三条：

1. 挑一件“只有你能干”的活，写下它的边界：到哪儿为止，什么不归它管。
2. 给它补三行设计决策：为什么这么设计、哪里看着奇怪但不能改。
3. 问自己一句：你请一个月假，它会不会停摆？会停在哪一步，那个“谁”知不知道自己是“谁”？

第 19 章 尾声（上）：所谓合格

一、盖了章，也只管今天

装修完验收，监理拿着一沓单子，一项一项打勾，最后盖一个章：合格。

你问他，这房子能住二十年吗？他说他只负责今天。

“合格”这两个字，大概就是这样一种东西：它证明的不是“以后不会出问题”，是“这次查的时候问题不大”。它给的是一个状态，不是一张终身有效的证书。

1998 年，我第一次写代码；2001 年正式入行。算到现在，年头已经很长了。

这些年我一直想给自己盖那个章。学会用框架的时候想盖一个，能改框架的时候想盖一个，造出自己的框架、被人用起来的时候又想盖一个。每次章还没盖稳，标准就往前挪了一格。

二、合格不是线，是状态

所以我想说的判断是：

合格不是一条线，是一种状态。

线是别人画的：考试六十分及格，面试过了发 offer，版本上线就算交付。这些线都很清楚，也很容易骗人——过了线，你就以为自己到了。

状态是自己维持的：今天这套东西还能不能被人接手，今天这个改动还有没有退路，今天写的这行代码，三个月后的自己还认不认得。它每天都可能掉下去，也每天都能捡回来。

这条路上最难的一关，不是“我够不够强”，是“我该在哪里停手”。

我前半程一直在加：加功能、加框架、加生成器、加一层又一层。加的时候很痛快，因为每一层都能看见东西变多。真正难的是减——减掉一个没人用的分支，减掉一个只有我能改的设计，减掉一次“顺手改一下”的冲动。

三、换使用者的眼睛看自己

如果非要给“合格”找一条分界线，我现在只认这一条：

分界线不在你会写多少，而在你什么时候开始用使用者的眼睛看自己写的东西。

前面那些事——“只有我能改”、把开源当成公开代码、看不起官网报表这类辅助系统——都是同一个盲点：一直站在作者的位置看系统，没站在用的人的位置看。

站在作者的位置，一切都很顺：我知道入口在哪，我知道这个字段为什么这么写，我知道出了问题先看哪。站在使用者的位置，这些“我知道”全都变成了他的“我不知道”。

这一条我不打算写成墙上的话。它只是一个动作：做任何东西之前，先写下“用户要完成的那一件事”，再看自己的界面、接口、文档，有没有绕开那件事。

这个动作我当时也做得不好。第一次认真做它，是在 redis helper 上：我想加一个“批量删除”，先写“用户要完成的那一件事”——把一堆没用的 key 清掉，别删错，删完还找得回来。写完才发现，我原来想的只是“选中、然后删”。后来补的是回收站，还有排序、首页统计、性能评估；界面基本没动。这个例子说明的只是：换到使用者的位置，看到的往往是功能，不是皮肤。

四、今天下午就能做的三件事

如果你想在这个迭代里就开始，我只推荐三件事，都不用等到“有空的时候”：

一、读一个扩展点。 挑你天天用的那个框架，找一个它留给你的扩展点，看它是在哪一步被调用的。看懂一个就够——你会第一次感觉到，“框架在调我”这句话具体长什么样。

二、抽一个工具类。 找那段你复制过三次的代码，抽成一个小东西，带默认值、带一行日志、带一个失败时的退路。

三、写一页理想中的调用示例。 不用实现，就写你希望别人怎么用它：五行代码，传什么、拿到什么、出错怎么办。写完你会发现，很多设计问题在“怎么被调用”这一页上就暴露了。

再加一句止损判断：**这件事如果只有你能做，就先别把它推给更多人用。** 先留入口，再谈推广。这条不是道理，是我用很多年换来的。

五、账我到现在也没算完

回到开头那个问题：我合格了吗？

如果把合格理解成“造出了一个成功的框架”，那我大概不合格——它让我忙了很多年，也把我拖到过很难看的地方。

但如果合格是“想明白了框架该造到什么程度、该放掉什么”，那我算是摸到了边。

还有一句话，我记了很多年。一位圈内权威人士总结经验，说程序员顶多混 30 年就结束了。

我当时的想法不是这句话对不对，而是：如果程序员坚持开发 30 年，会怎么样？既然大家都从这条路上撤退，我继续走，会不会反倒是个机会？

一开始，我的各项天赋能力都落在人后。我就想，能不能专攻一行——挑大家觉得基础、没什么难点的事，一直做下去。

我是这么想的，也是这么做的。于是这些年，我看到了一路的风景，遇到了不少志同道合的朋友，一起享受开发的快乐，也享受协作的快乐。

第 1 章里我说过差不多的话：那年我以为，学会用别人的框架就是终点；很多年后才知道，造东西的人要付什么账。

账我到现在也没算完。只是我终于知道，哪些账本来就不该记在自己头上；还有些账，记了也还不上，只能先挂在心里。

对人这一侧，我想明白了一半。对机器那一侧，还有一件更具体的事。

本章的一句话：合格不是一条线，是一种状态；最难的不是变强，是知道在哪里停手。

今天能用的三条：

1. 读一个你天天用的框架的扩展点，看它是在哪一步被调用的。
2. 把复制过三次的那段代码抽出来，补上默认值、一行日志、一条退路。
3. 写一页“我希望别人怎么用”的调用示例，五行代码，写完再回头看设计。

第 20 章 尾声（下）：让英文去编译，让中文去解释

Let English compile. Let Chinese explain.

一、双语，真不是因为英语差

先说个误会。

有人听说我写代码用双语，第一反应是：“英语不太好的人，才这么干吧。”

我理解。这就像看见一个人吃饭配两个碗，先猜他胃不好。真实情况是：我这个年纪的程序员，英语学得不算差，也不算好，读文档靠猜、写注释靠蒙，属于那种“能看懂，但不敢保证自己看懂了”的水平。所以我最早的双语习惯，确实有点怂——中文写一遍，怕自己以后看不懂；英文再写一遍，怕别人看不懂。两个都写，图个安心。

但后来我发现，这事跟英语水平关系不大。真正的问题不在“用哪种语言写”，而在**写下来的东西到底是给谁看的**。

二、注释是写给三个月后的自己

我先说一句可能不太受欢迎的话：

大部分项目里，注释不是写给人看的，是写给“三个月后的自己”看的，而那个自己，通常也看不懂。

不是字不认识，是不知道当时为什么这么写。你写过这种注释吗——“// 这里做特殊处理”。特殊在哪儿？处理什么？为什么不能正常处理？三个月后你自己都答不上来。

这就是我想说的那句：

让英文去编译，让中文去解释。

Let English compile. Let Chinese explain.

这句话不是语言偏好，是分工。英文那部分，是机器要读的东西：标识符、目录结构、接口名、提交信息、给 AI 的提示词。机器读它，编译它，检索它，不

需要情感。中文那部分，是人要读的东西：为什么这么写、当时在怕什么、哪条业务规则不能碰。人读它，是为了接手，不是为了背诵。

一句话：**给机器好编译的，给人好理解的。** 别让机器去猜中文语义，也别让人去猜英文缩写。

这句话不是哪天忽然想出来的。它是被一堆麻烦一点点逼出来的——所以第三节不先讲道理，先讲那些麻烦。不讲清楚它们，这句话就只是一句听着挺顺的口号。

到这里，如果你觉得“这不就是注释规范吗”，那我说得还不够。因为它不是一个写作习惯，是我这套框架里最硬的一层东西。

三、先有代码，后有规则

我这套框架的代码生成，不是设计出来的，是**改规则改出来的**。

一开始只是嫌重复：同样的增删改查、同样的列表页、同样的表单校验，一个项目抄一遍，抄到第三遍就烦了。于是写个小工具，把能自动的部分自动掉——那时候的“规则”，其实是“我这几天懒得写哪几段”。它不严谨，但有效。

后来前端从 Ext JS 那套换成 bootstrap 那一层，麻烦立刻显形：生成器里写死的东西太多，前端一换，生成出来的代码全得回头改。**规则没写清楚的地方，就是换技术栈时要还债的地方。** 那段时间我不是在写新功能，是在一条一条把“我当时顺手那么写”翻译成“以后必须这么写”。

再往后，AI 进来了。AI 有一个好处，也有一个坏处。好处是它不怕重复，你给它规则，它能一直照做；坏处是它只认**写得清楚的规则**——你含糊，它就自由发挥，发挥得还挺自信。

于是这套框架的核心规则，是在跟 AI 来回折腾的过程中一条一条定下来的。不是我先想好一套完美规范再落到代码里，而是反过来：**先有代码，再有反复的修改，最后才有规则。** 说准确点是：这套东西迭代了十几年，直到 AI 到来之后，**核心规则的定义才算完成**——不是因为我终于想清楚了，是因为机器把“说不清楚的地方”全部暴露出来了。

所以我现在敢说一句更准确的话：

框架的核心不是代码，是一套被反复修正的规则。代码只是规则的执行结果。

这条规则能被翻出来，不是因为我记性好，是因为它落在三个地方：README、框架代码、生成工具。三个层级都带着它，它才算真的定住。

那这条规则是什么时候写下来的？答案有点丢人：很晚。

我把仓库从头翻了一遍。这句条文第一次出现在 README，是 2026 年 7 月的一次文档提交，整段是新增的，不是从哪段旧话里改出来的。也就是说，在它被写下来之前，没有“上一版条文”——只有一堆早就这么写的代码。

代码早到什么时候？2016 年主仓第一次提交里，控制器就已经是英文类名、英文方法名，注释是中文；生成器那时候也已经带着“表和列必须有注释”的校验。再往前那套 Ext JS 的老代码，逐条历史已经查不到了，能看到的只有一份快照——也是同一个样子。

所以顺序是反的：我先这么写了很多年，才想起来把它写成一句话。不是条文规定了实践，是实践攒到一定程度，条文才被补上。

（可复核：`a2c10c11` 2026-07-27 README 首次写入；`0c9b1a0` 2016-11-16 主仓首提代码仍是这个形态；`8efb7dc5` 2017-07-10 生成器已把中文注释拼进英文骨架。）

代码会过时，框架会换壳，语言会换——Rust 也好 React 也好，都只是当时的工具。但“这条规则为什么存在、它挡住的是哪类错误”，这个东西能留下来。而它留下来的前提是：**它被写下来了，而且写得别人能读懂。**

这就接上了前面几章。第 18 章说，能不能交出去，取决于知识能不能被别人读懂。第 19 章说，要用使用者的眼睛看自己写的东西。这一章补最后一块：**能被写下来的规则，才是能被别人接手的东西。** 你脑子里的经验，再值钱，也是你一个人的；写成规则，才是团队的。

四、双语是要付维护费的

但这句话有边界，不说清楚就是口号。

第一，双语是要付维护费的。 中英两份，一定会不一致。代码改了注释没改，英文名改了中文说明没改——这种时候必须有一条裁决顺序，否则两份都不可信。

我的做法是：**以代码为准，注释过期就删掉重写，不留“大概是这个意思”的残留**。宁可没有注释，不要有一句骗人的注释。

第二，团队的语言习惯不是一致的。 有人英文读得慢，有人中文写得别扭。硬推一套规范，最先反弹的往往是干活最多的人。所以规则要少，要能一句话说清，要允许“这条我做不到，但我保证标识符是英文”这种降级。

第三，面向国际读者的开源项目，中文注释是障碍。 这个我认。所以对外开源的部分，注释尽量英文，中文只留在内部文档和提交说明里——不是不写中文，是不把中文塞进别人读不到上下文的地方。

这条规则不是嘴上说说。它在这套框架里落地了三处。

README 里是明文：**"Code in English, Comment in Chinese."（注释用中文，代码用英语。）** 这条约定到今天还在更新（README 最近一次改动是 2026 年 10 月 22 日），说明它不是一个当年的口号，而是一份一直有人维护的东西。

代码里没有条文，只有习惯。控制器 `Action_Auth` 是英文名，方法叫 `logout()`、`login()` 也是英文，注释却是 `/** 退出 */`、`/** 登录 */`。一个文件这么写不稀奇，几百个文件都这么写，它就不再是“规范”，而是默认动作。

生成工具里最硬。它从数据库的表注释、列注释里取出中文，直接拼进生成的英文代码骨架：`/** 显示{中文注释} */` 配 `public static String xxxShow(...)`。另外还有一道校验：表和列没有注释，生成时直接报错（`nocomment`、`column_nocomment`）。管得是有点宽，但它至少保证了一件事——英文那半边，永远有一句中文兜着。

顺便说个实话：生成器里并没有把这条规则写成一句条文。我把它的文件翻了一遍，没找到这句原文。它是靠默认模板和中英搭配落地的，真正的明文只在 README 里。这也挺像我们这行的作风：规则写得最清楚的地方，往往不是代码。

（说明：以上路径与提交均来自本机仓库只读核对，仓库未做任何改动。）

第四，也是最要紧的一条：这不是永久真理。 AI 对中文的理解一直在变好。今天要求“英文给机器、中文给人”，是因为机器读中文还不稳、读英文更稳。等它哪天读中文和读英文一样稳了，这条规则的形态一定会变。**规则是有保质期的，写规则的人得知道自己在什么阶段。**

五、条文只有一句

所以真正能带走的东西，不是“双语”，是一条能直接贴在项目里的条文。

这条规则是我自己的做法，写下来就一句：

```
Code in English, Comment in Chinese.
```

先把这两层关系说清楚，免得看起来像两个不同的主张：前面那句是**领会**——为什么这么分；这一句是**条文**——怎么落到每一次提交里。同一件事的两层，不是两句话。领会管方向，条文管执行。

条文很硬，一句话，不留解释空间：代码用英文，注释用中文。但条文之外，还有三处实现细节，那是我给的建议，不是条文的一部分：

1. **标识符、接口名、提交信息，用英文。** 一句话理由：这些东西要被机器检索、被别人搜索、被 AI 索引，英文的命中率更高。
2. **注释只写“为什么”，用中文。** “做了什么”让代码自己说；注释里只留业务约束、坑、以及“改了会出什么事”。写不出“为什么”，就别写注释。
3. **给 AI 的提示词，写清输入、输出、约束三件事。** 别描述愿景，描述边界：给它什么、要它吐什么格式、哪些绝对不能动。

（这三条是围绕条文展开的实现建议，不是条文本身——条文只有一句。你还要往下延伸——文档、设计说明、提交规范各自用哪种语言——判断标准只有一条：**下一个接手的人能不能读懂。** 具体分到哪种语言，按你团队的实际来，不必照抄。）

这些都很小，小到不像“方法论”。但它们恰好是那种今天下午就能改、改完下个月还在生效的东西。

六、留下的不是代码

最后说个实话。

我年轻时以为，一个程序员的水平体现在他写了多少代码。后来觉得，体现在他造出了什么东西。再后来，被自己造的东西拖垮过一次，才明白：**你留下的不是代码，是别人能不能在你不在场的时候，继续把这件事做对。**

代码会烂，框架会换，语言会过时。规则如果写清楚了，它能活得比这些都久一点。

让英文去编译，让中文去解释。

Let English compile. Let Chinese explain.

作者简介

雪碧天 著

华东师范大学教育技术学毕业，不是计算机科班。1998 年在学校和一家公司合办的实验室里开始写代码，做多媒体、Flash 和网站；2001 年正式入行，之后一直在上海做开发。

写过业务，也读过框架源码，后来自己造了一个开源框架 betterlife，GitHub 上是 skygreen2001。

这本书不教你成功。它记录一个人怎么一边写代码，一边把坑一个个踩成经验——踩了二十多年，坑还没踩完。

光缘志

我以为我本是一只飞蛾，原来我承蒙了神的恩典成就为人。

神说要有光，于是我就成了光；光就是我，我就是光；

后来神创造了人，我就成了人；而我相信光、遇见光、成为光。

于是成为光的我，要全心全意为人民服务。